

8

Lists in Python

8.1 INTRODUCTION

Sequences arise naturally in many real-life situations. A year has 12 months (January, February, March, April, May, June, July, August, September, October, November and December). The months of the year are in a sequence. After January, it will always be February. It is impossible to jump from March to July because it has to go in a sequence. So, we say a **Sequence** is an object that contains multiple items of data. The items are stored in a sequence one after the other such as a sequence of events, movements, items, or a sequence in which a player records (music) with a sound recorder/sequencer. Consider the example of a sequence of cars as shown in Fig. 8.1.

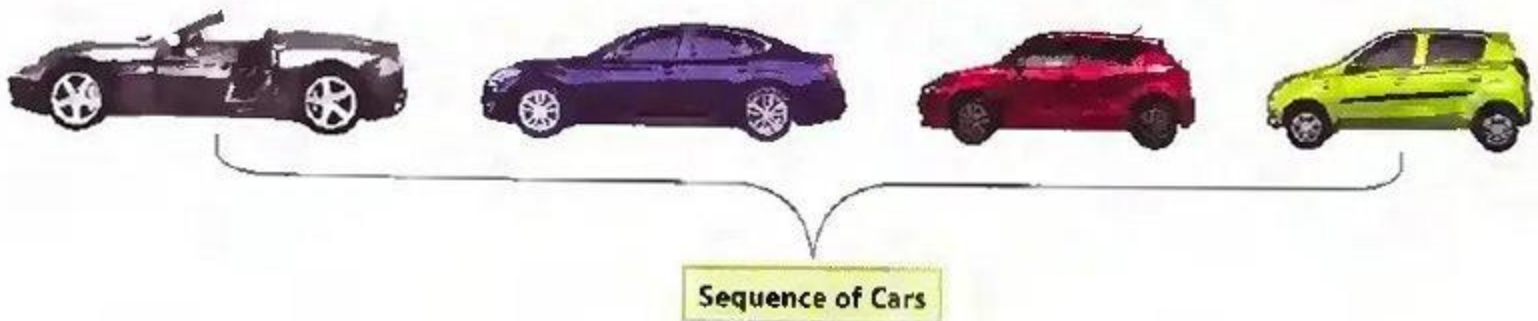


Fig. 8.1: A Sequence of Cars

As is evident from the above figure, there is a sequence of cars and each car becomes an element/item of this particular sequence. Here, we have a grey-coloured car, a blue car, a red car and so on. So, all of these cars combined together form a sequence. Python also supports sequence which can be considered as containers and in those containers certain items are stored. A sequence may have repeated items in the list. The number of elements is called the length of the sequence. The various sequences available in Python are shown in Fig. 8.2.

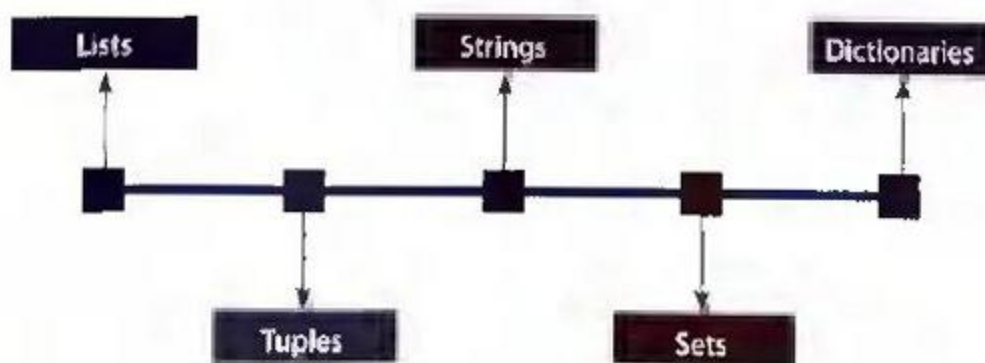


Fig. 8.2: Sequences in Python

In this chapter, we will focus only on Lists and rest of the Python sequences shall be discussed in the successive chapters.

8.2 LIST

A list is a collection of values or an ordered sequence of values/items. The items in a list can be of any type such as string, integers, float, objects, or even a list. The values that make up a list are called its elements. Elements of a list are enclosed in **square brackets []**, separated by commas. Lists allow duplicate values as well. The element of the list can be accessed by its index. Lists are mutable, i.e., values in the list can be modified, which means that Python will not create a new list when you make changes to an element of a list. Elements in a list need not be of the same type. In other words, we say that lists are heterogeneous (of multiple types) in nature.

Like a string (sequence of characters), list also is a sequenced data type.

CTM: A list is a collection of comma-separated values (items) within square brackets. Items in a list need not be of the same type.

8.2.1 Declaring/Creating/Initializing List

Lists can be created by putting comma-separated values/items within square brackets []. Square brackets indicate start and end of the list, consisting of elements separated by a comma. The syntax for creating a list is:

```
<list_name> = []
```

For example,

```
L = []
```

This construct is termed as a **"List Display Construct"**. Square brackets in the above syntax contain the set of values/elements constituting the list with the name given as <list_name>. This is termed as **initialization** of the list with a blank or no values or we can say an **empty list**. A list can be populated with any type of values/collection of items by enclosing inside square brackets [] as shown in the example given below:

For example,

```
Fruits = ['Mango', 'Apple', 'Grapes']
```

In the above example, we have a list of fruits having three items—Mango, Apple and Grapes, separated by comma and enclosed inside square [] brackets. Other examples of lists are:

List Types and Examples:

Lists in Python can be broadly classified into three types:

1. Empty Lists
2. Long Lists
3. Nested Lists

Learning Tip: Make sure that strings are enclosed in single/double quotes. Python can't differentiate between single or double quotes. So, we can use whatever we want.

<code>list1=[10, 20, 30, 40, 50]</code>	<code>#list of Integers</code>
<code>list2=["Deep", 450, 30.4, "Python", -2200, True, False]</code>	<code>#list with elements of #different data types</code>
<code>list3=['A', 'E', 'I', 'O', 'U']</code>	<code>#list of characters</code>
<code>list4=[]</code>	<code>#empty list, i.e., list with no element</code>
<code>list5=["Delhi", "Mumbai", "Chennai"]</code>	<code>#list of 3 strings</code>
<code>list6=[3, 4, [5,6,7], 8]</code>	<code>#list containing another list as an element, i.e., nested lists</code>
<code>list7=[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625]</code>	<code>#long list, i.e., a list #containing #many elements which are #squares of numbers starting #from 0 to 25</code>

```

Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list1=[10,20,30,40,50]      #list of integers
>>> list1
[10, 20, 30, 40, 50]
>>> list2=["Deep",450,30.4,"Python",-2200,True,False]  #list with elements of different
data types
>>> list2
['Deep', 450, 30.4, 'Python', -2200, True, False]
>>> list3=['A','E','I','O','U']    #list of characters
>>> list3
['A', 'E', 'I', 'O', 'U']
>>> list4=[]      #empty list, i.e., list with no element
>>> list4
[]
>>> list5=["Delhi","Mumbai","Chennai"]  #list of 3 strings
>>> list5
['Delhi', 'Mumbai', 'Chennai']
>>> list6=[3,4,[5,6,7],8]  #list containing another list as an element,i.e.,Nested list
>>> list6
[3, 4, [5, 6, 7], 8]
Ln: 22 Col: 4

```

```

>>> list7=[0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225,256,289,324,361,400,441,48
4,529,576,625]
>>> list7=[0,1,4,9,16,25,36,49,64,81,100,121,144,169,196,225,256,289,324,361,400,441,48
4,529,576,625]  #long list,i.e., a list containing many elements which are squares of
numbers starting from 0 to 25
>>> list7
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361,
400, 441, 484, 529, 576, 625]

```

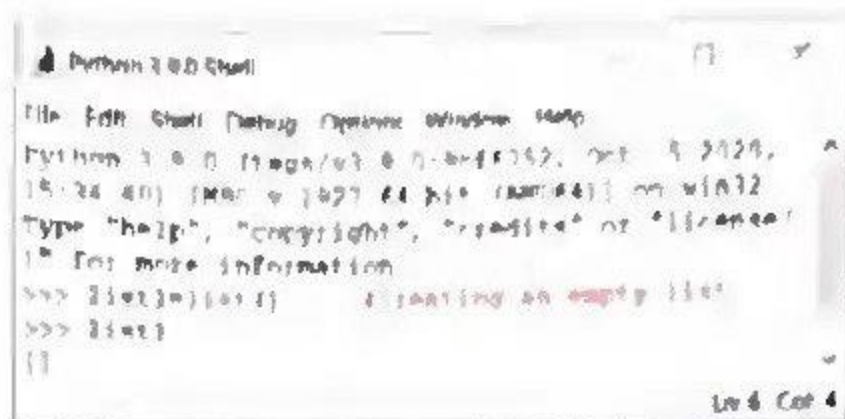
Fig. 8.3: Creating Different Types of Lists

These lists can be created in Python by typing in front of Python shell prompt `>>>` and displaying its contents by simply typing the list name as shown in Fig. 8.3.

Creating a List from an existing sequence

The `list()` method in Python takes sequence types and converts a given record (tuple or string) into list. We can create a list from an existing sequence using this built-in function in the following manner:

1. An empty list created with built-in function `list()` Fig. 8.4(a)



```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:24:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>> list()
[]
```

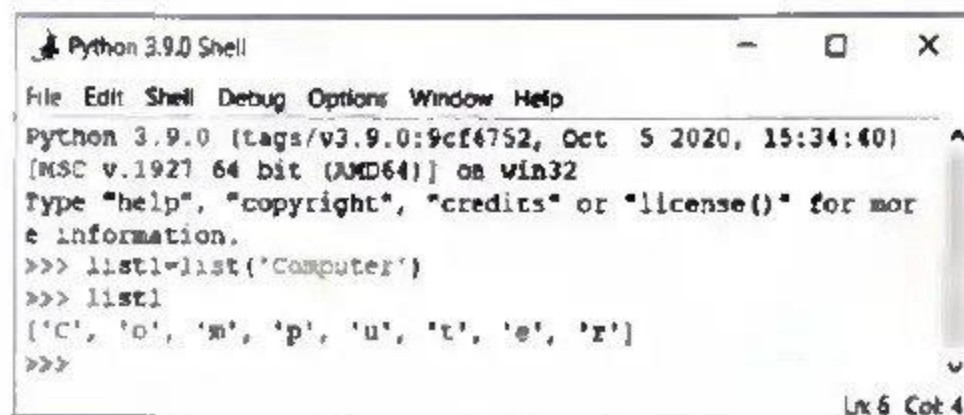
Fig. 8.4(a): Creating List using `list()` Function

2. Creating a list from a sequence (string):

`new_list_name = list(sequence/string)`

For example,

```
>>> list1 = list('Computer')
list1
['C', 'o', 'm', 'p', 'u', 't', 'e', 'r']
```

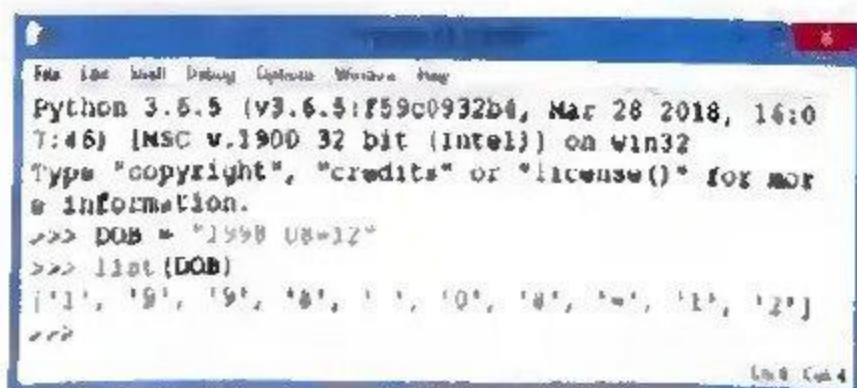


```
Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> list1 = list('Computer')
>>> list1
['C', 'o', 'm', 'p', 'u', 't', 'e', 'r']
>>>
```

Fig. 8.4(b): Creating a Sequence into a List using `list()` Function

In the given example, a new list, `list1` is created from a sequence 'Computer' and displayed by typing the name of the list, i.e., `>>>list1` as shown in Fig. 8.4(b).

Another example is for accepting date of birth as input and converting it into a list using `list` method as shown in Fig. 8.4(c).

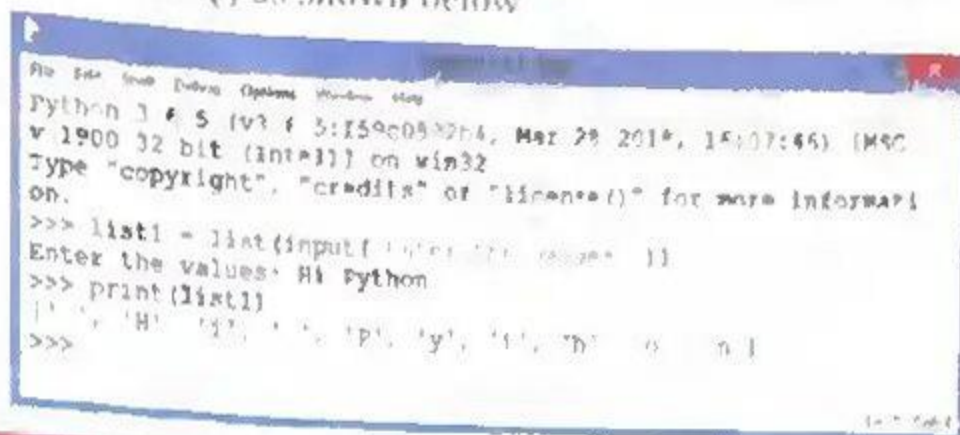


```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> DOB = "1998 08-12"
>>> list(DOB)
['1', '9', '9', '8', ' ', '0', '8', '-', '1', '2']
>>>
```

Fig. 8.4(c): Converting Date of Birth into a List using `list()` Function

3. Creating a list through user input using list() method

You can create a new list by extracting the elements from the user through command prompt and using built-in method list() as shown below



```
Python 3.5.4 (v3.5.4:309274, Mar 29 2016, 15:07:46) [MSC
v1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
>>> list1 = list(input("Enter the values: "))
Enter the values: Hi Python
>>> print(list1)
['H', 'i', ' ', 'P', 'y', 't', 'h', 'o', 'n']
>>>
```

POINTS TO REMEMBER

- print() function can be used to display an entire list.
- list() function can convert certain types of objects into lists.

> Creating a List from an existing list

You can create a new list from an already existing list as follows:

Consider a list,

```
>>> list1 = [10, 20, 30, 40, 50]
```

Now, we shall be creating new lists by extracting values from this list.

Python statement	Description
list5 = list1[:]	list5 is a copy of list1. list5 = [10, 20, 30, 40, 50]
list6 = list1[1:4]	list6 shall contain elements 1, 2, 3 from list1. list6 = [20, 30, 40]
list7 = list1	list7 shall have all the elements present in list1. list7 = [10, 20, 30, 40, 50]

8.2.2 Accessing List Elements (Indexing)

A list consists of any collection of items which are stored according to its index. Like strings any item in a list can be accessed by using the slice operator[] by specifying its index value.

List indices start with 0 (zero) and go to length -1. The list index can be a positive or negative integer value or an expression which evaluates to an integer value. Hence, indexing in lists can be positive or negative, i.e., you can access the elements from a list either moving from left to right (positive index) or from right to left (negative index). The index of -1 refers to the last item, -2 to the second last item and so on.

For example, consider list1 containing 10 elements.

```
list1 = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

This list in computer memory shall be represented as shown in Fig. 8.5 below:

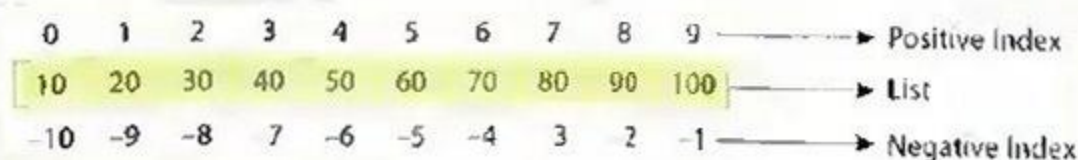


Fig. 8.5: Positive and Negative Indices in a List

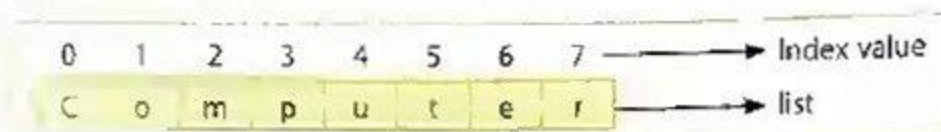
Each element of the above list can be accessed through their indexes enclosed inside square brackets.

```

Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v
.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more informatio
n.
>>> list1 = [10,20,30,40,50,60,70,80,90,100]
>>> list1
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
>>> list1[0]    index 0, i.e., 10
10
>>> list1[6]    index 6, i.e., 70
70
>>> list1[9]    index 9, i.e., 100
100
>>> list1[-10]  index -10, i.e., 10
10
>>> list1[-5]   index -5, i.e., 60
60
>>>

```

Consider another list with elements, 'C', 'o', 'm', 'p', 'u', 't', 'e', 'r' stored inside memory as shown:



```

>>> list=['C','o','m','p','u','t','e','r']
>>> list[3]
'p'

```

```

===== RESTART: Shell =====
>>> list = ['C','o','m','p','u','t','e','r']
>>> list[3]
'p'
>>>

```

- An *IndexError* appears if you try and access an element that does not exist in the list. It will display the error message as: *IndexError: list index out of range*

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Type "copyright", "credits" or "license()" for more info
>>> list = ['C','o','m','p','u','t','e','r']
>>> list[8]
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    list[8]
IndexError: list index out of range
>>>

```

Now, in the above code, we are trying to access eighth element (`>>>list[8]`) from the list which does not exist as we are having a total of 8 elements for which the last index is 7. Since we are trying to access the element at index 8, Python will give an "IndexError".

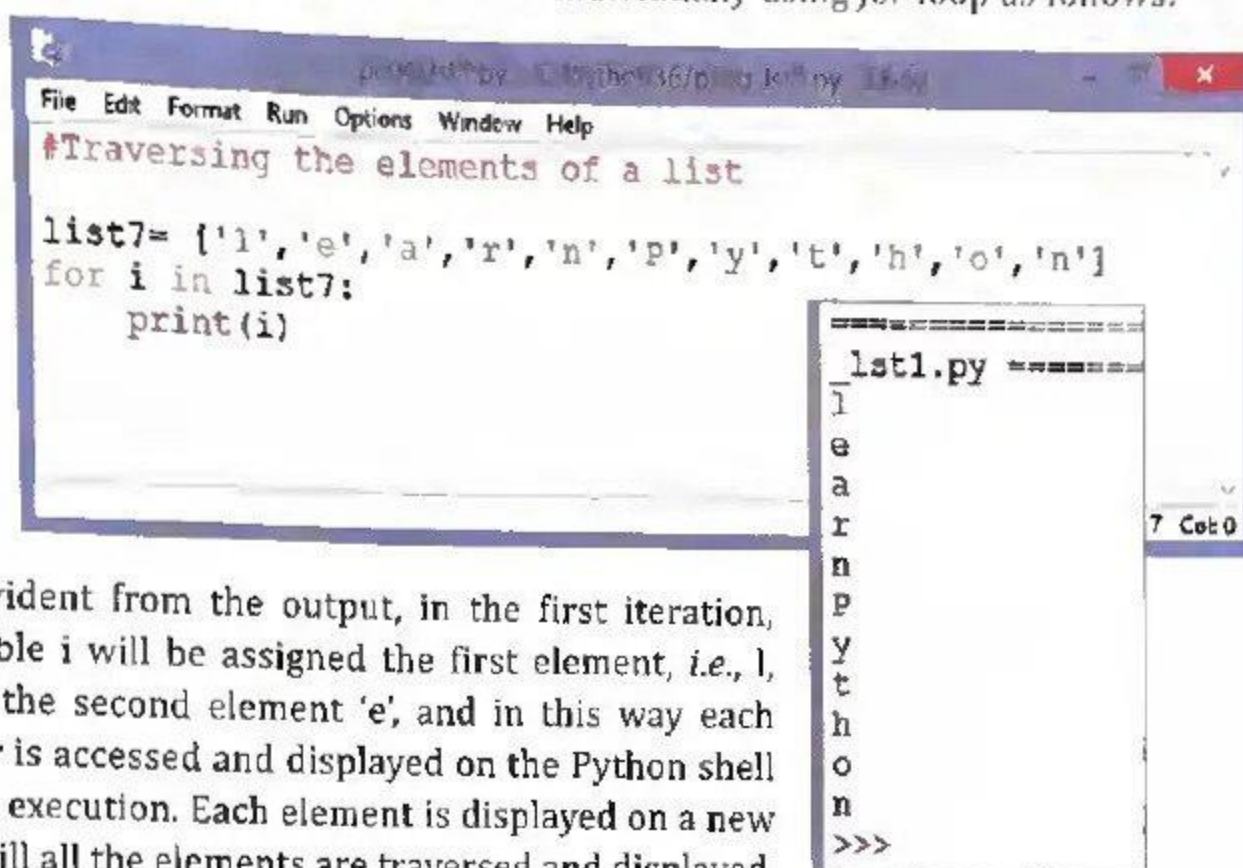
For example, `my_list = ['p', 'r', 'o', 'b', 'e']`
`print(my_list[-1])` Output: e
`print(my_list[-5])` Output: p

8.2.3 Traversing a List

Traversing a list means accessing each element of a list. This can be done by using either *for* or *while* looping statement. Traversing can be done in two ways:

1. Using 'in' operator inside *for* loop

The elements of a list can be accessed individually using *for* loop as follows:



The screenshot shows a Python IDE window titled 'lst1.py'. The code in the editor is:

```
#Traversing the elements of a list  
list7= ['l', 'e', 'a', 'r', 'n', 'i', 'n', 'g', 't', 'h', 'o', 'n']  
for i in list7:  
    print(i)
```

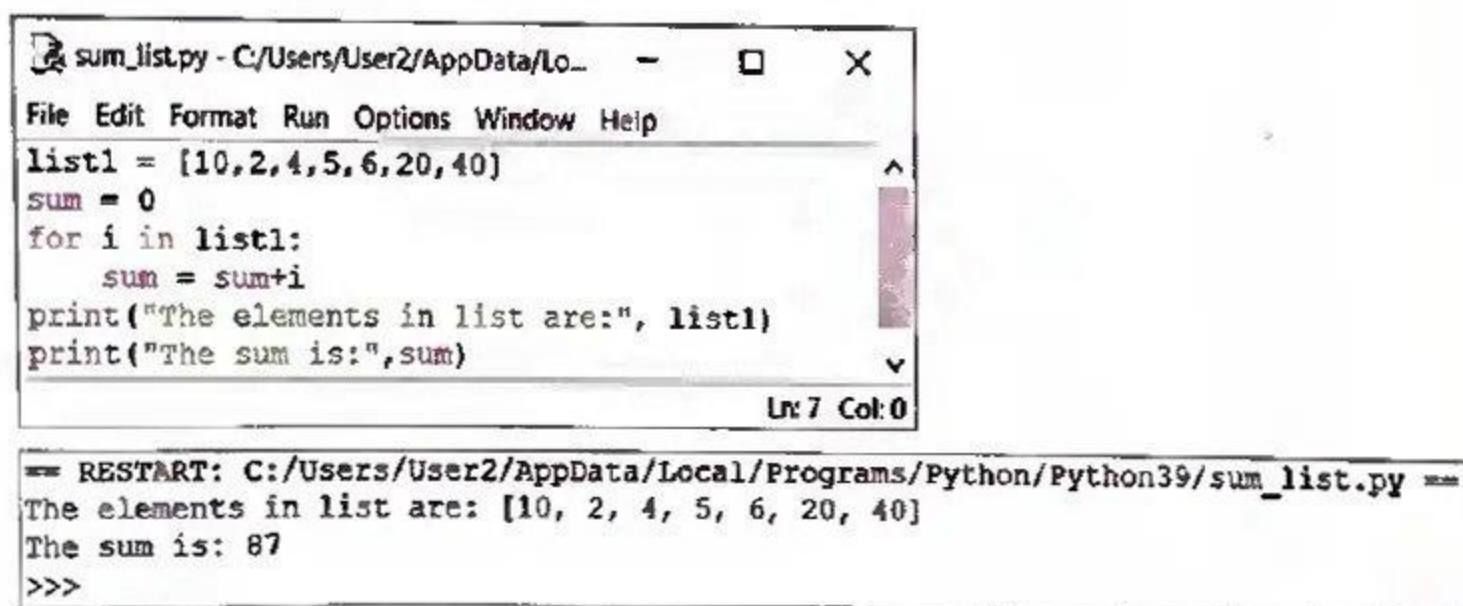
The output window on the right shows the following output:

```
l  
e  
a  
r  
n  
i  
n  
g  
t  
h  
o  
n  
>>>
```

As evident from the output, in the first iteration, variable *i* will be assigned the first element, i.e., *l*, then the second element 'e', and in this way each letter is accessed and displayed on the Python shell upon execution. Each element is displayed on a new line till all the elements are traversed and displayed.

Here, 'in' operator using *for* loop iterates each element of a list in sequence.

Example 1: To find the sum of all elements in a list.



The screenshot shows a Python IDE window titled 'sum_list.py'. The code in the editor is:

```
list1 = [10, 2, 4, 5, 6, 20, 40]  
sum = 0  
for i in list1:  
    sum = sum+i  
print("The elements in list are:", list1)  
print("The sum is:", sum)
```

The output window shows the following output:

```
== RESTART: C:/Users/User2/AppData/Local/Programs/Python/Python39/sum_list.py ==  
The elements in list are: [10, 2, 4, 5, 6, 20, 40]  
The sum is: 87  
>>>
```

2. Using `range()` function

We have already discussed this function in Chapter 6. It checks for the given set of values mentioned as the argument to `range()` function. This function can be used for accessing individual elements of a list, using indexes of elements only, along with `len()` method which gives length of the list and specifies for how many times *for* loop will iterate.

Example 2: To count the total number of characters in a string.

```

File Edit Format Run Options Window Help
list1 = ['P','y','t','h','o','n']

n = len(list1)  #len() counts the total number of
                #characters in a string

for i in range(n):
    print(list1[i])
print("Total number of characters are: ")
print(n)

>>>
===== RESTART: C:\python36\prog
py =====
P
y
t
h
o
n
Total number of characters are:
6
>>>

```

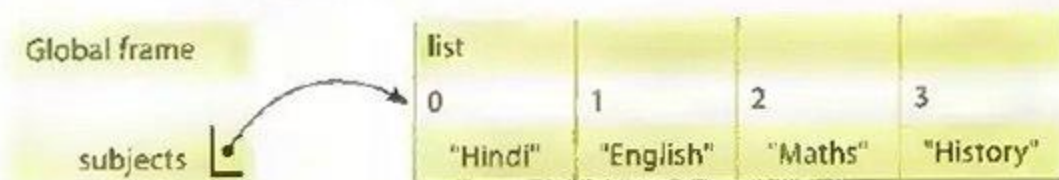
8.2.4 Aliasing

In Python, variables refer to objects. The lists we create in Python are also objects we are referring to. Since lists are mutable, they can be modified. If we assign the elements of one list to another list, both shall refer to the same object.

For example,

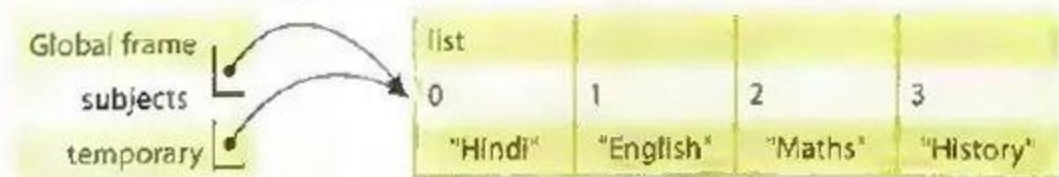
Let us create a list named subjects containing names of four subjects as string type.

```
>>> subjects= ['Hindi', 'English', 'Maths', 'History']
```



The elements of subjects list in memory are represented according to their index values. Since the list index always starts with 0, so Hindi is stored at index value 0, English at index 1, Maths at index 2, and History at index 3. Now we will assign it to create a temporary list of the already created list, subjects, by giving the statement as:

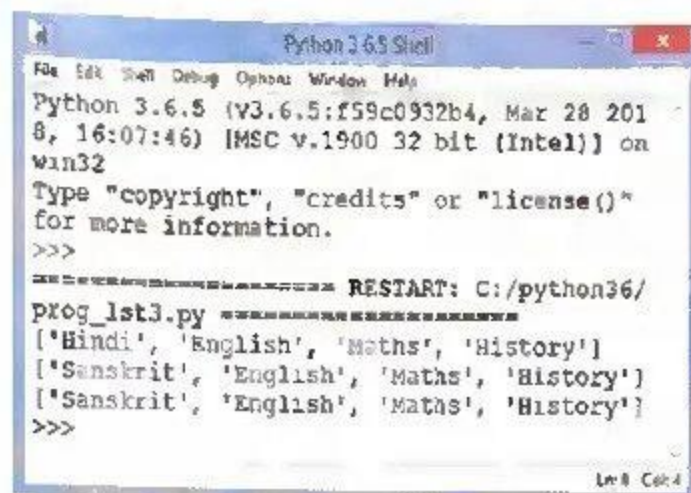
```
>>> temporary = subjects
```



Now we have two lists having the same set of values. In other words, temporary is an alias (alternate) name for subjects list as assignment with an '=' operator does not make a copy, instead makes both these lists referring to the same location. We can modify the contents of these lists by referring to its individual index value.

Suppose we have to modify 'temporary' list by changing the subject 'Hindi' to 'Sanskrit', which is stored at index 0, by giving the command as:

```
>>> temporary[0] = 'Sanskrit'
>>> print(temporary)
>>> print(subjects)
['Sanskrit', 'English', 'Maths', 'History']
['Sanskrit', 'English', 'Maths', 'History']
```



```
Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/python36/ =====
prog_lst3.py
['Hindi', 'English', 'Maths', 'History']
['Sanskrit', 'English', 'Maths', 'History']
['Sanskrit', 'English', 'Maths', 'History']
>>>
```

It is evident from the given example that the modifications made in the temporary list are reflected in the subjects (original) list as well since the same list is having two different names. So, we can say that changes made with one alias get reflected in the other alias.

Although this behaviour can be useful, yet it is sometimes unexpected or undesirable, so aliasing should be avoided.

The above list can be modified and represented with two values at one index position.

```
>>> subjectCodes = [['Sanskrit', 43], ['English', 85], ['Maths', 65],
                    ['History', 36]]

>>> subjectCodes[1]
['English', 85]
>>> print(subjectCodes[1][0], subjectCodes[1][1])
English 85
```

In the above example, the list is represented in two-dimensional manner. Therefore, in order to access their individual element from the nested list, we have to mention the row and column index value.

8.2.5 Comparing Lists

Python allows us to compare two lists. Each element is individually compared in lexicographical (alphabetical or dictionary) order. We can apply all the relational operators (>, <, ==, !=, etc.) to compare two lists if they are of comparable types, otherwise Python flashes an error.

For example,

Consider the following lists,

```
>>> L1, L2 = [10, 20, 30], [10, 20, 30]
>>> L3 = [10, [20, 30]] #Nested list
>>> L1 == L2
True
>>> L1 == L3
False
```



As shown in the given example, the second statement evaluates to False because L3 is a nested list and L1 is a simple list. Thus, on comparison it generates False because of not being comparable types.

Further, Python generates the final result of non-equality comparisons as either True/False from corresponding elements' comparison. While comparing two lists, it checks element-by-element comparison. If the corresponding elements are equal, it goes on to the next element and so on, until it finds the element that differs. Subsequent elements are not considered even if they are big as shown in the example given below:

Comparison	Result	Reason
<code>[1,2,3,4]<[4,5,6]</code>	True	<code>1<4</code> True
<code>[1,2,3,4]<[1,5,2,3]</code>	True	<code>2<5</code> True
<code>[1,2,3,4]>[1,2,0,3]</code>	True	<code>3>0</code> True
<code>[1,2,3,4]<[1,2,3,2]</code>	False	<code>4<2</code> False

In the above example, on comparing the first, second and third elements, it yields True, but on comparing the fourth element, 4, which is not less than 2, it generates False and, hence, the output becomes False.

Therefore, for comparing two lists, Python internally compares individual elements of two lists, applying all the comparison rules. Consider the following example:

```
>>> l1= [20,30]
>>> l2= [20,30]
>>> l3= ['20','30']
>>> l4= [20.0,30.0]
>>> l5= [20,30,40]
>>> l1 == l2
True
>>> l1 == l3
False
>>> l1 > l2
False
>>> l4 > l1
False
>>> l4 == l1
True
>>> l1 < l5
True
```

For comparison operations, Python ignores the types of elements and compares values only.

In the above example, the second statement (`l1 == l3`) results in False value since the types of both the lists are dissimilar. In the third statement, the two lists are similar and, hence, greater than (`l1 > l2`) operator results in False. But when we compare the two lists, l1 and l4, which are of integer type and float type respectively, it results in True. This is because Python *int* and *float* with matching values are considered equal and, hence, the output becomes True. Therefore, for two lists to be equal, they must have the same number of elements and matching values.

Example: To find at least one common element amongst the two inputted lists.

```
list_check.py - C:\Users\User2\AppData\Loc...
File Edit Format Run Options Window Help
list1 = [1,100,200,3,5,400]
list2 = [2,10,300,4,5,500]
print("The elements in list1 are:",list1)
print("The elements in list2 are:",list2)
for x in list1:
    for y in list2:
        if x == y:
            print("The common element is:",x)
```

```
= RESTART: C:/Users/User2/AppData/Local/Programs/Python/Python39/list_check.py =
The elements in list1 are: [1, 100, 200, 3, 5, 400]
The elements in list2 are: [2, 10, 300, 4, 5, 500]
The common element is: 5
>>>
```

8.3 OPERATIONS ON LISTS

Python provides us with several basic operations which can be performed on lists as shown in Fig 8.6.



Fig. 8.6: Common Operations on Lists

Let us discuss some of the important operations using an example:

```
list1 = ['Red', 'Green']
list2 = [10, 20, 30]
```

8.3.1 Concatenation

Concatenation or joining is a process in which multiple sequences/lists can be combined together with the help of certain operators. Python allows combining or adding of two lists using '+' concatenation operator (addition operation). We can add two lists using this operator.

```

prog_lst4.py - C:/python36/prog_lst4.py (3.6.5)
File Edit Format Run Options Window Help
# List concatenation using '+' operator
# adding two strings
list1= ['Red','Green']

list1= list1 + ['Blue']
print(list1)

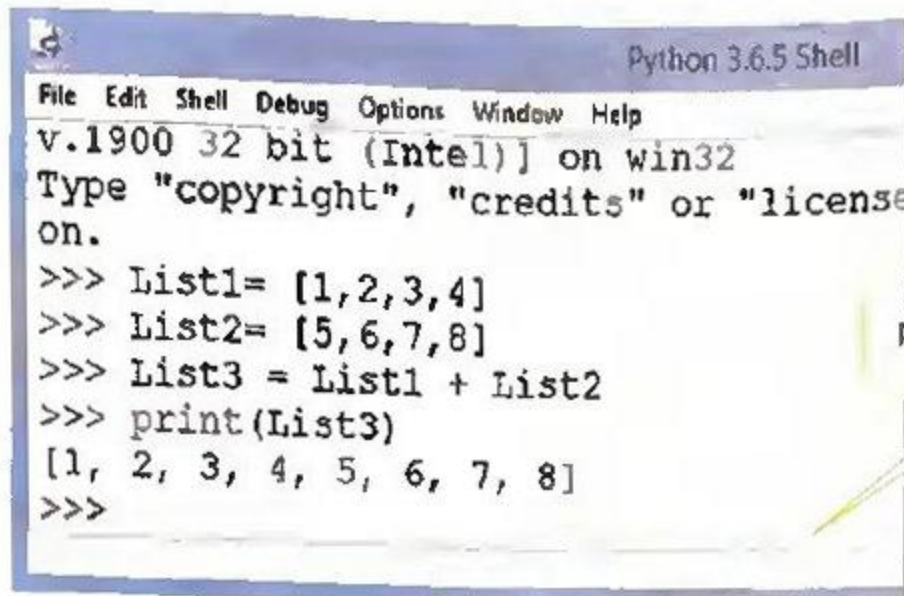
#adding two lists
list= ['learn','Python'] + ['Programming']
print(list)

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 20
18, 16:07:46) [MSC v.1900 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()"
for more information.
>>>
===== RESTART: C:/python36
/prog_lst4.py =====
['Red', 'Green', 'Blue']
['learn', 'Python', 'Programming']
>>>
Ln:7 Col:4
  
```

```
>>> List1 = [1, 2, 3, 4]
```

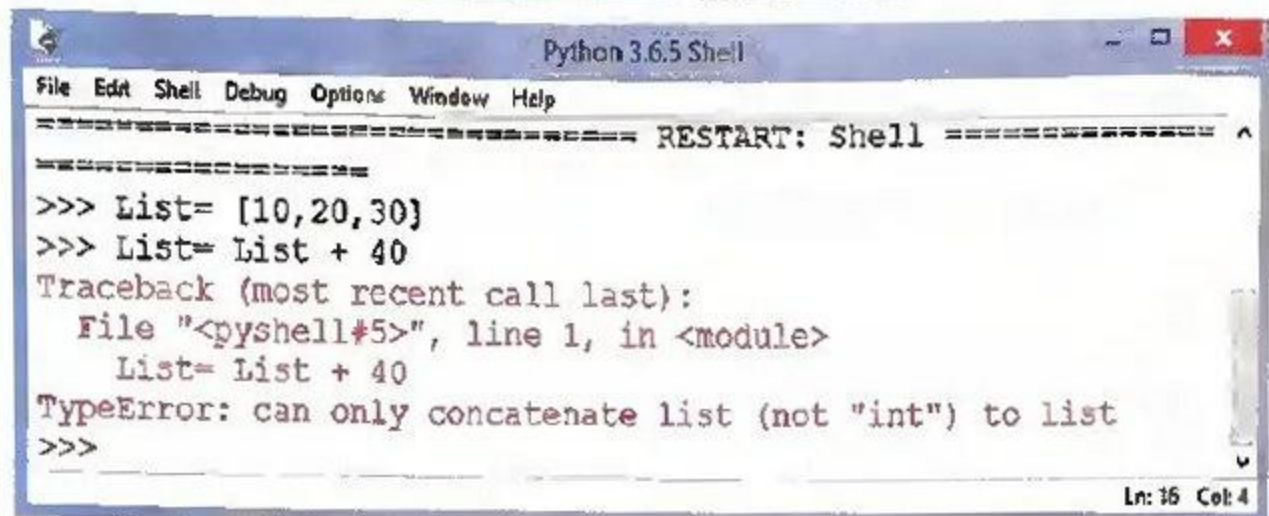
```
>>> List2 = [5, 6, 7, 8]
```

#the + operator concatenates lists:



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
v.1900 32 bit (Intel) on win32
Type "copyright", "credits" or "license()"
>>> List1 = [1, 2, 3, 4]
>>> List2 = [5, 6, 7, 8]
>>> List3 = List1 + List2
>>> print(List3)
[1, 2, 3, 4, 5, 6, 7, 8]
>>>
```

Note: '+' operator simply performs a concatenation with list.



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
===== RESTART: Shell =====
>>> List = [10, 20, 30]
>>> List = List + 40
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    List = List + 40
TypeError: can only concatenate list (not "int") to list
>>>
```

In the above example, the statement: `List = List + 40` shall generate an error since concatenation operation can be performed between two lists. So, when we try to add a numeric value 40 to this list, Python will generate a `TypeError` as shown in the above screenshot.

```
>>> List = [10, 20, 30]
>>> List + "hello"
Traceback (most recent call last):
  File "<pyshell#19>", line 1, in <module>
    List + "hello"
TypeError: can only concatenate list (not "str") to list
```

Similarly, if we try to concatenate a list with string value, it will again generate `TypeError`. Thus, we conclude that the '+' operator, when used with lists, requires both the operands to be of list types.

8.3.2 Repetition/Replication

Multiply (* asterisk) operator replicates the List for a specified number of times and creates a new list.

```
>>> list2 = [10, 20, 30]
```

```
>>> list2 * 2
```

```
[10, 20, 30, 10, 20, 30]
```

```

C:\python36\prog_lst5.py
File Edit Shell Debug Options Window Help
#Repetition operation
list2 = [10,20,30]
print(list2*2)

x= 'Python' * 3
print(x)

```

```

Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\python36\prog_lst5.py =====
>>>
[10, 20, 30, 10, 20, 30]
PythonPythonPython
>>>

```

Fig. 8.7(a): Replicating a List

In the above example, list2 is multiplied by 2 and string 'Python' is multiplied by 3. Thus, list is printed two times and Python gets printed three times (Fig. 8.7(a)). However, we cannot multiply one list with another as shown in Fig. 8.7(b).

```

Python 3.6.5 Shell
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> DOB = "1998-08-12"
>>> list(DOB)
['1', '9', '9', '8', '-', '0', '8', '-', '1', '2']
>>> list = [1,2,3,4]
>>> list * list
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    list * list
TypeError: can't multiply sequence by non-int of type 'list'
>>>

```

Fig. 8.7(b): List Multiplication by another List Generates Error

8.3.3 Membership Testing

Membership testing is an operation carried out to check or test whether a particular element/item is a member of that sequence or not.

As you can see in Fig. 8.8(a), the car on the left-hand side does not belong to the sequence of cars, whereas the car on the right belongs to this sequence. Hence, we say that the left-hand side car is not a member and the right-hand side car is a member of the sequence of cars.

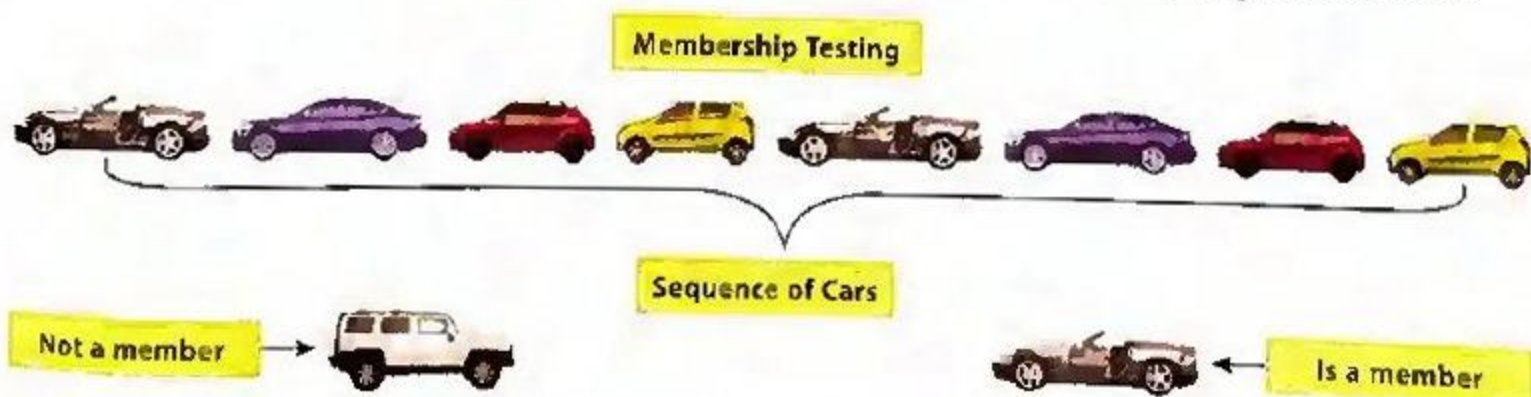
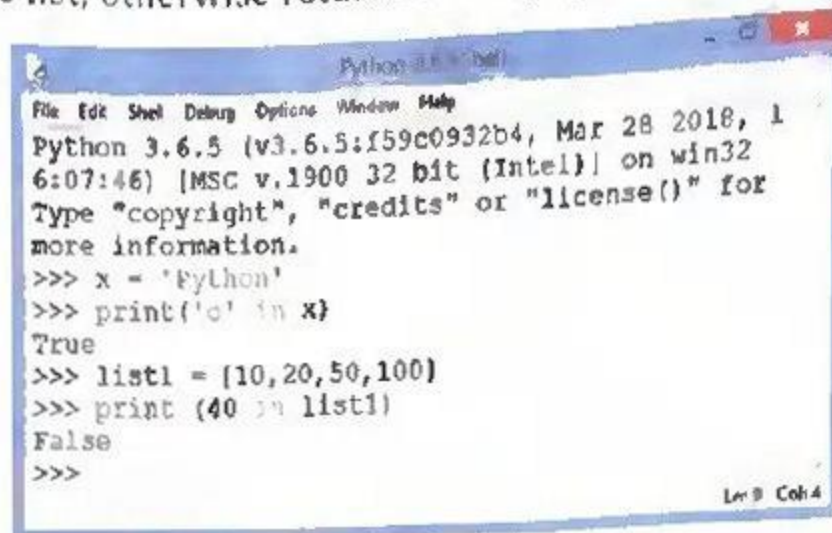


Fig. 8.8(a): Membership Testing

Python also allows us to check for membership of individual characters or substrings in strings using 'in' operator.

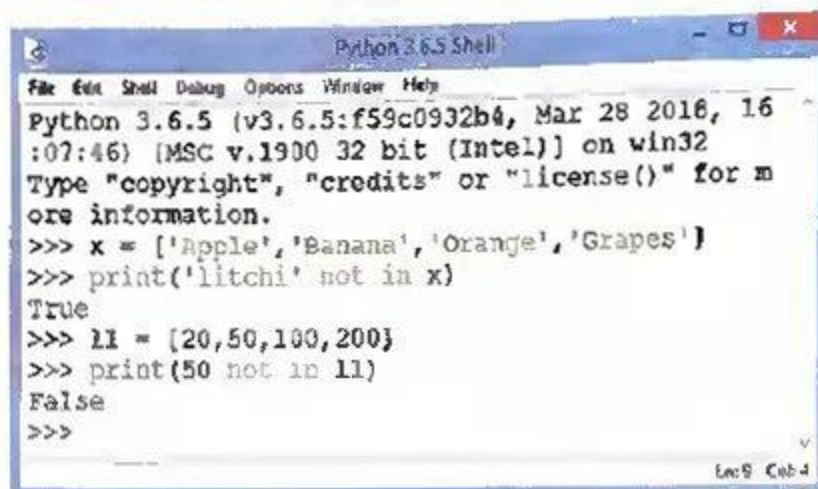
The 'in' operator checks whether a given element is contained in a list. It returns True if the element appears in the list, otherwise returns False (Fig. 8.8(b)).



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = 'Python'
>>> print('o' in x)
True
>>> list1 = [10,20,50,100]
>>> print(40 in list1)
False
>>>
```

Fig. 8.8(b): Membership Operator 'in'

The reverse of 'in' is 'not in'. 'not in' operator returns True if the element does not appear in the list, otherwise returns False (Fig. 8.8(c)).



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> x = ['Apple', 'Banana', 'Orange', 'Grapes']
>>> print('litchi' not in x)
True
>>> l1 = [20,50,100,200]
>>> print(50 not in l1)
False
>>>
```

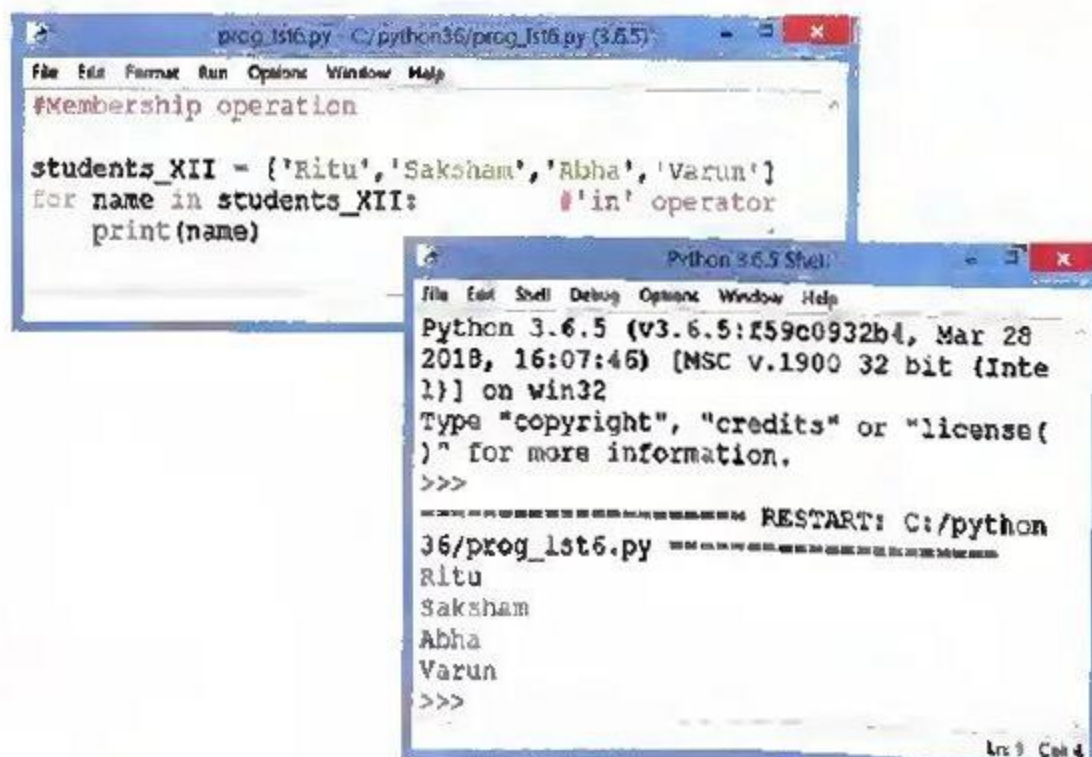
Fig. 8.8(c): Membership Operator 'not in'

➤ 'in' operator used with for loop

In this method, the membership operator *in* can be used to display the elements of a list through *for* loop (Fig. 8.8(d)).

For example,

To display the names of students who are members of class XII.



```
prog_1st6.py - C:/python36/prog_1st6.py (3.6.5)
File Edit Format Run Options Window Help
#Membership operation

students_XII = ['Ritu', 'Saksham', 'Abha', 'Varun']
for name in students_XII:
    print(name)
```

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/python36/prog_1st6.py
Ritu
Saksham
Abha
Varun
>>>
```

Fig. 8.8(d): 'in' Operator using for Loop

8.3.4 Indexing

Index is nothing but there is an index value for each item present in the sequence. So, you have 0th index, 1st index, 2nd index, 3rd index, and so on, depending upon the sequence that you have. In Python, indexing starts from 0 like other programming languages. As shown in the example of sequence of cars in Fig. 8.9(a), we have 8 items in this particular sequence and the index range is from 0 to 7. Thus, lists in Python can have/store different types of elements stored in a particular sequence such as integers, strings, etc.

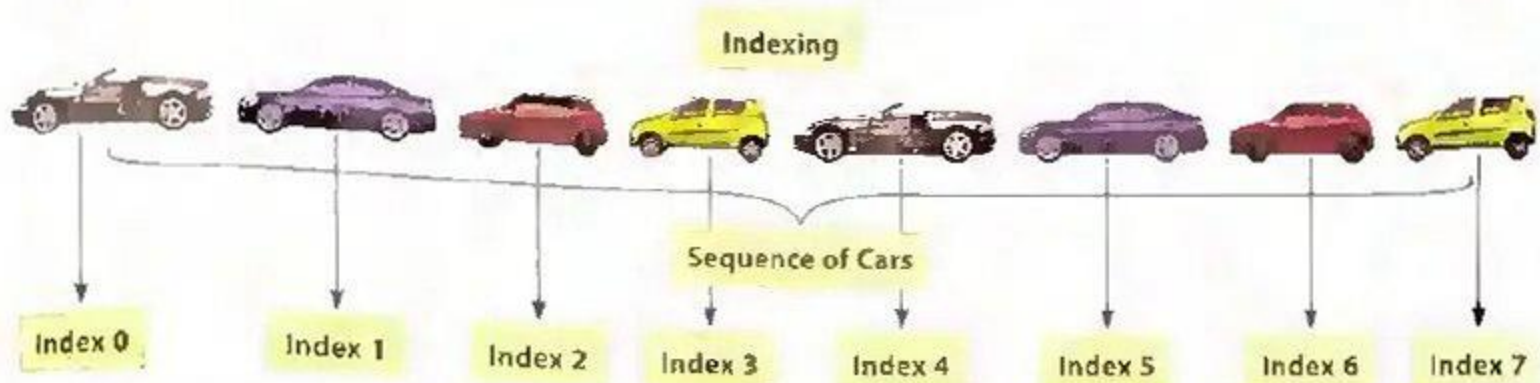


Fig. 8.9(a): Indexing

Similarly, in Python, an index is a number specifying the position of an element in a list. It enables access to individual element in the list. Index of first element in the list is 0, second element is 1, and n^{th} element is $n-1$.

Negative indexes identify positions relative to the end of the list. The index -1 identifies the last element, -2 identifies next to the last element, etc., as shown in Fig. 8.9(b).

```
#Indexing operation

list1= ['Blue', 'Blue', '30', '30']
list2= [10, 20, 30, 40, 50]

print(list1[2])
print(list1[-1])
print(list2[2])
print(list2[-3])
```

```
>>>
===== RESTART: C:/pyth
on36/prog_1st7.py =====
>>>
Blue
Blue
30
30
>>>
```

Fig. 8.9(b): Indexing

8.3.5 Slicing

Indexing and slicing are two inter-related operations in lists. Slicing is done through indexing. Let us first understand what slicing is.

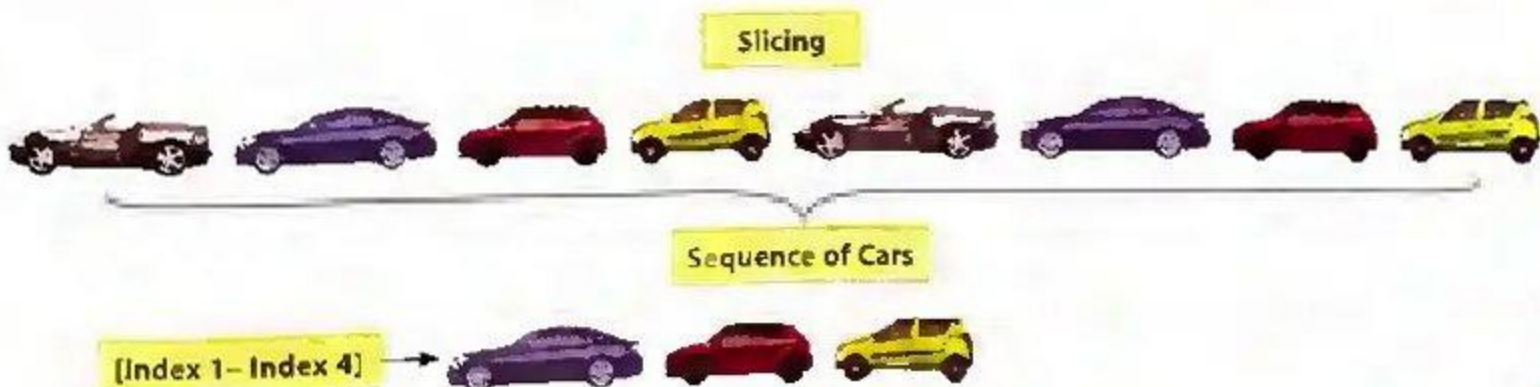


Fig. 8.10: Slicing Operation



Slicing is an operation in which we can slice a particular range from that sequence. In the context of Fig. 8.10, imagine you want to access only two or three cars (starting with blue car) out of four cars given as a sequence. So, the range to be taken shall be from index 1 – index 4, where index 4 car is to be excluded. Now we will see this operation on lists.

List slices are the sub-parts of a list extracted out. List slices can be created through the use of indexes. Slicing is used to retrieve a subset of values. A slice of a list is basically its sub-list. While indexing is used to obtain individual items, slicing allows us to obtain a subset of items. When we enter a range that we want to extract, it is called range slicing.

The syntax used for slicing is:

Syntax:

```
list[start:stop:step]
```

where, start is the starting point; stop is the stopping point, which is not included; and step is the step size—also known as stride—and is optional. Its default value is 1.

To take an example, let us consider a list/sequence as `x='c','o','m','p','u','t','e','r'`. The various slicing operations performed on this sequence shall generate the following set of items as shown in the table given below:

Code	Result	Explanation
<code>x[1:4]</code>	<code>['o','m','p']</code>	Items 1 to 3
<code>x[1:6:2]</code>	<code>['o','p','t']</code>	Items 1, 3, 5
<code>x[3:]</code>	<code>['p','u','t','e','r']</code>	Items 3 to end
<code>x[:5]</code>	<code>['c','o','m','p','u']</code>	Items 0 to 4
<code>x[-1]</code>	<code>['r']</code>	Last item
<code>x[-3:]</code>	<code>['t','e','r']</code>	Last 3 items
<code>x[:-2]</code>	<code>['c','o','m','p','u','t']</code>	All except last 2 items

Let us consider another list, 'list1', with the following values and results after slicing operation:

```
list1 = [100, 50.75, "Radhika", "Shaurya", 900, 897.60]
```

The screenshot shows a Python IDE window titled 'prog_list8.py - C:\python36\prog_list8.py (3.6.5)'. The code in the editor is:

```
#Slicing operation

list1 = [100, 50.75, "Radhika", "Shaurya", 900, 897.60]
print(list1[:])
```

Below the editor is a 'Python 3.6.5 Shell' window showing the execution output:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46)
(MSC v.1900 32 bit (Intel)) on win32
Type "copyright", "credits" or "license()" for more info
rmentation.
>>>
===== RESTART: C:\python36\prog_list8.py =====
[100, 50.75, 'Radhika', 'Shaurya', 900, 897.6]
>>>
```

```

1) on win32
Type "copyright", "credits" or "license()" for more information
>>> list1 = [100, 50.75, 'Radhika', 'Shaurya', 900]
>>> print(list1[2:5])
['Radhika', 'Shaurya', 900]
>>> print(list1[:5])
[100, 50.75, 'Radhika', 'Shaurya', 900]
>>> print(list1[3:])
['Shaurya', 900, 900]
>>> print(list1[:2])
[100, 'Radhika']
>>> print(list1[-2])
900
>>> print(list1[-5:-2])
[50.75, 'Shaurya', 900]
>>> print(list1[-5:-2])
[50.75, 'Radhika', 'Shaurya']
>>> print(list1[5:2:-1])
[900, 900, 'Shaurya']
>>> print(list1[-2:-5:-1])
[900, 'Shaurya', 'Radhika']
>>> print(list1[-2:-5:-2])
[900, 'Radhika']
>>>

```

In the above example, the list slicing is done by extracting the elements and printing it. It is to be noted here that in statements 7 to 9 (print statements), we have given step value. Lists also support **slice steps**. Therefore, if we wish to extract the elements, not consecutive but every other element of the list, we can do it using the **slice steps**.

This can also be done using a variable to store the elements retrieved using list slicing.

For example,

```

>>> list1 = [24, 56, 10, 99, 28, 90, 20, 22, 34]
>>> slice1 = list1[3:-3]
>>> slice1
[99, 28, 90]
>>> slice1[1]
28
>>> slice1[1] = 30
>>> slice1
[99, 30, 90]

```

As shown above, the slice extracted from list1 is stored in the variable, slice1. The list slice is a list in itself, i.e., we can perform all operations on it just like we perform on lists. Thus, when we try to modify the list slice, slice1, it is successfully implemented since the list slice, slice1, is a list in itself.

The slicing operation is entirely based on indexing. In case the resulting index lies outside the list, Python raises an **IndexError: list index out of range**. Slice range (start and stop argument) is treated as boundary, and the result will simply contain all the elements which lie between the boundary.

In case the start and stop limits lie outside the boundary, Python still returns the elements that lie between the specified range/boundary without raising any error.

```

>>> list1 = [24, 56, 10, 99, 28, 90, 20, 22, 34]
>>> list1[3:30]
[99, 28, 90, 20, 22, 34]
>>> list1[-15: 7]
[24, 56, 10, 99, 28, 90, 20]
>>> list1[2:6]
[10, 99, 28, 90]
>>> list1[5:10]
[90, 20, 22, 34]
>>> list1[6:10] #Stop limit is out of bound
[20, 22, 34]
>>> list1[8:15]
[34]
>>> list1[10:20] #Both limits are out of bounds
[]

```

In the last statement, both the limits are out of bounds, still Python does not give any error but returns an empty list as no element of list1 possesses the index that falls in the range of 10 to 20.

CTM: In lists slicing, if start index value is missing then it will start from 0 by default and if stop value is missing then it will print till end. Stop index, if given, then it will stop at index stop-1, i.e., stop is excluded.

8.4 NESTED LISTS

When a list appears as an element of another list, it is called a nested list, i.e., a list can have one or more lists as its elements.

```

>>> list1 = [1, 2, 'a', 'c', [6, 7, 8], 4, 9]
>>> #fifth element of list is also a list
>>> list1[4]
[6, 7, 8]

```

To access the element of nested list of list1, we have to specify two indices list1[i][j]. The first index i will take us to the desired nested list and the second index j will take us to the desired element in that nested list.

```

>>> list1[4][1]
7

```

The output shown above is generated as 7 because index i gives the fifth element of list1 which is 4 passed as the argument to i and index j gives the second element in the nested list, with argument as 1.

```

>>> list1=[1,2,3,'a',['apple','green'],5,6,7,['red','orange']]
>>> list1[4][1]
'green'
>>> list1[6]
6
>>> list1[8][0][2]
'd'
>>> list1[4:6]
[['apple', 'green'], 5]
>>> list1[4]='mango'
>>> list1
[1, 2, 3, 'a', 'mango', 5, 6, 7, ['red', 'orange']]
>>> list1[-1]
['red', 'orange']
>>> list1[8][0]='black'
>>> list1
[1, 2, 3, 'a', 'mango', 5, 6, 7, ('black', 'orange')]
>>> len(list1)
9
>>>

```

8.5 COPYING LISTS

Given a list, the simplest way to make a copy of the list is to assign it to another list.

```

>>> list1 = [1, 2, 3]
>>> list2 = list1
>>> list1
[1, 2, 3]
>>> list2
[1, 2, 3]

```

The statement `list2 = list1` does not create a new list. Rather, it just makes `list1` and `list2` refer to the same list object. Here, `list2` actually becomes an alias of `list1`. Therefore, any changes made to either of them will be reflected in the other list.

```
>>> list1.append(10)
>>> list1
[1, 2, 3, 10]
>>> list2
[1, 2, 3, 10]
```

We can also create a copy or clone of the list as a distinct object by three methods. The first method uses slicing, the second method uses built-in function `list()` and the third method uses `copy()` function of Python library.

Method 1: We can slice our original list and store it into a new list variable as follows:

```
newList = oldList[:]
```

```
>>> list1 = [1, 2, 3, 4, 5]
>>> list2 = list1[:]
>>> list2
[1, 2, 3, 4, 5]
```

Method 2: We can use the list constructor function `list()` as follows:

```
newList = list(oldList)
```

```
>>> list1 = [10, 20, 30, 40]
>>> list2 = list(list1)
>>> list2
[10, 20, 30, 40]
```

Calling it without any arguments will create an empty list.

Method 3: We can use the `copy()` function as follows:

```
newList = list(oldList).copy()
```

```
>>> list1=[1,2,3,4,5]
>>> list2=list1.copy()
>>> list2
[1, 2, 3, 4, 5]
>>> list1[0]=100
>>> list1
[100, 2, 3, 4, 5]
>>> list2
[1, 2, 3, 4, 5]
>>> list2[2]=300
>>> list2
[1, 2, 300, 4, 5]
>>> list1
[100, 2, 3, 4, 5]
```

The `copy()` function returns a new list stored in a different memory location. It doesn't modify the original list or the modifications made in the new list will not be reflected to the base list.

8.6 BUILT-IN FUNCTIONS (MANIPULATING LISTS)

Python offers several built-in 'in-place' functions that can alter the elements of the list rather than creating a new list, such as adding new elements, changing the elements in a list, etc. These functions perform various operations on lists which are described in Fig. 8.11.

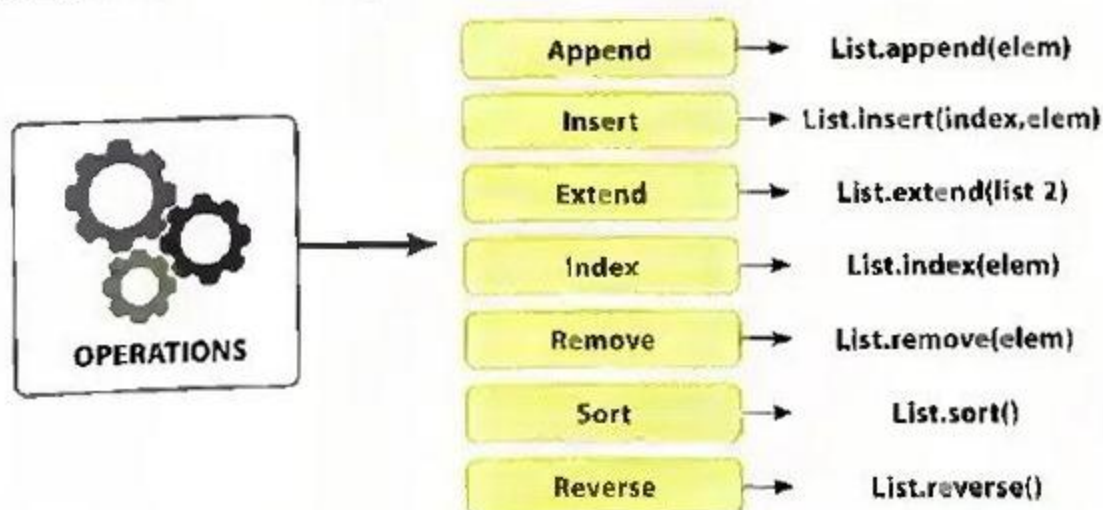


Fig. 8.11: Built-in List operations

8.6.1 append()

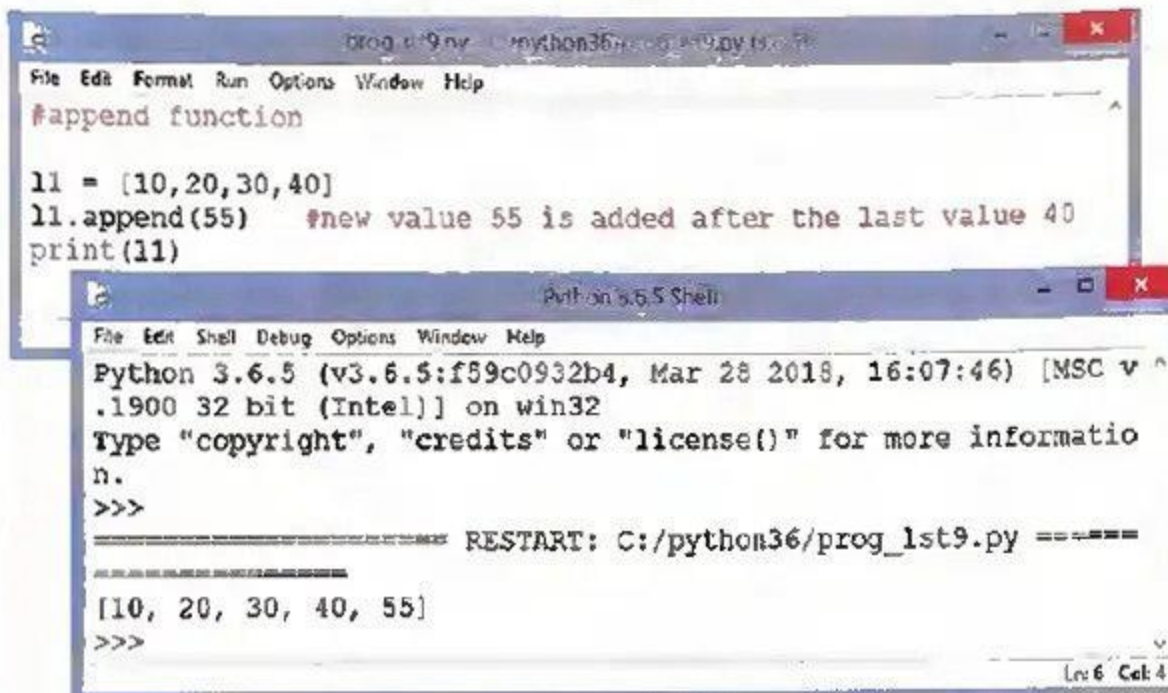
The `append()` method adds a single item to the end of the list. It doesn't create a new list; rather it modifies the original list. The single element can also be a list. The syntax of `append()` method is:

Syntax:

```
list.append(item)
```

The item can be numbers, strings, another list, dictionary, etc.

Example 3:



The screenshot shows two windows from a Python IDE. The top window, titled 'prog_lst9.py', contains the following code:

```
#append function  
  
l1 = [10,20,30,40]  
l1.append(55)    #new value 55 is added after the last value 40  
print(l1)
```

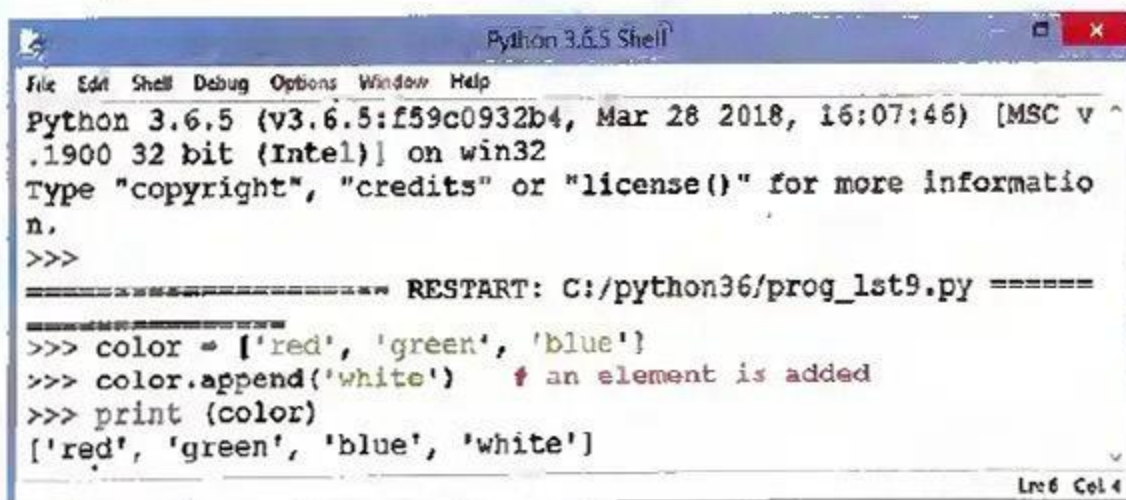
The bottom window, titled 'Python 3.6.5 Shell', shows the execution output:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v ^  
.1900 32 bit (Intel)] on win32  
Type "copyright", "credits" or "license()" for more informatio  
n.  
>>>  
===== RESTART: C:/python36/prog_lst9.py =====  
[10, 20, 30, 40, 55]  
>>>
```

A list can also be added to the above existing list. *For example,*

```
>>> l1 = [10,20,30,40]  
>>> l1.append([50,60])  
>>> l1  
[10, 20, 30, 40, [50, 60]]
```

Example 4:



The screenshot shows a 'Python 3.6.5 Shell' window with the following code and output:

```
>>> color = ['red', 'green', 'blue']  
>>> color.append('white')    # an element is added  
>>> print (color)  
['red', 'green', 'blue', 'white']
```

Practical Implementation-1

Write a program to find the second largest element in a list Num

```
#Program to find the second largest element in a list Num
print("Enter the total number of elements in the list:")
N = int(input())
i = 0
num = []
while i < N:
    print("Enter element", i, ":")
    num1 = int(input())
    num.append(num1)
    i += 1
print("The list is:", num)
for i in range(N):
    print(num[i], end=" ")
    if (num[0] > num[1]):
        m, m2 = num[0], num[1]
    else:
        m, m2 = num[1], num[0]
    x = num[i+2:]
    if x > m2:
        if x > m:
            m2, m = m, x
        else:
            m2 = x
print()
print("The second largest element in the list is:", m2)
```

Practical Implementation-2

Give Python code to accept values from the user up to a certain limit entered by the user. If the number is even, then add it to a list.

```
list_even_ch-7_CS.py - C:/Users/User2/AppData/Local/...
File Edit Format Run Options Window Help
L=[]
n=int(input("Enter total number of values in list:"))
i=1
while i<=n:
    a=int(input("Enter elements:"))
    if a%2==0:
        L.append(a)
    i=i+1
print(L)
Ln 10 Col 0
```

*****Output of the program*****

```
= RESTART: C:/Users/User2/AppData/Local/Programs/Python/Python39/list_even_ch-7_
CS.py
Enter total number of values in list:4
Enter elements:10
Enter elements:5
Enter elements:8
Enter elements:3
Enter elements:6
Enter elements:2
[10, 8, 6, 2]
>>>
```

8.6.2 extend()

The `extend()` method adds one list at the end of another list. In other words, all the items of a list are added at the end of an already created list.

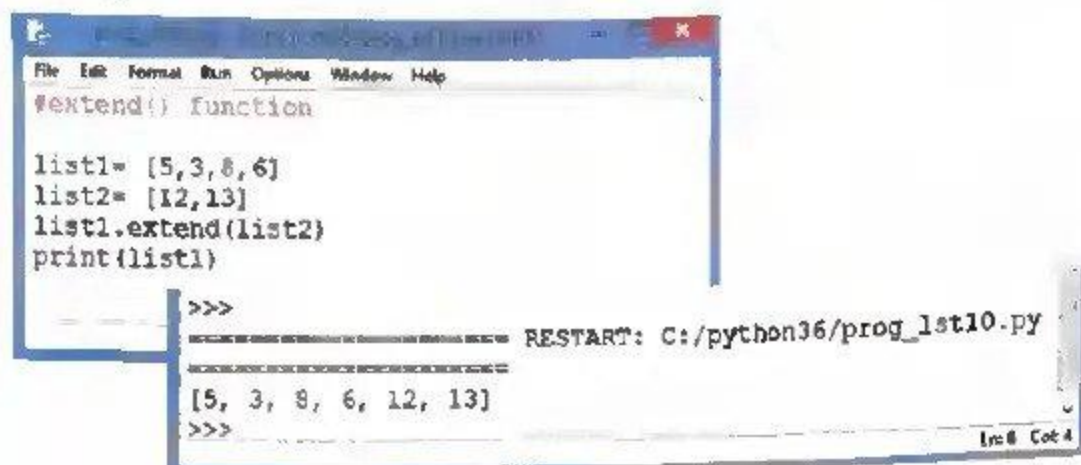
Syntax:

`list1.extend(list2)`

→ The list which will be added to list1.

→ list1 is the primary list which will be extended.

Example 5:



```
#extend() function

list1= [5,3,8,6]
list2= [12,13]
list1.extend(list2)
print(list1)

>>>
===== RESTART: C:/python36/prog_lst10.py
[5, 3, 8, 6, 12, 13]
>>>
```

Example 6:

```
>>> L1=[100,200,300,400] → list1
>>> L2=[10,20,30] → list2
>>> L1.extend(L2)
>>> print(L1)
[100, 200, 300, 400, 10, 20, 30] → items of list2 added to list1
```

8.6.3 insert()

The `insert()` function can be used to insert an element/object at a specified index. This function takes two arguments: the index where an item/object is to be inserted and the item/element itself.

Syntax:

`list_name.insert(index_number, value)`

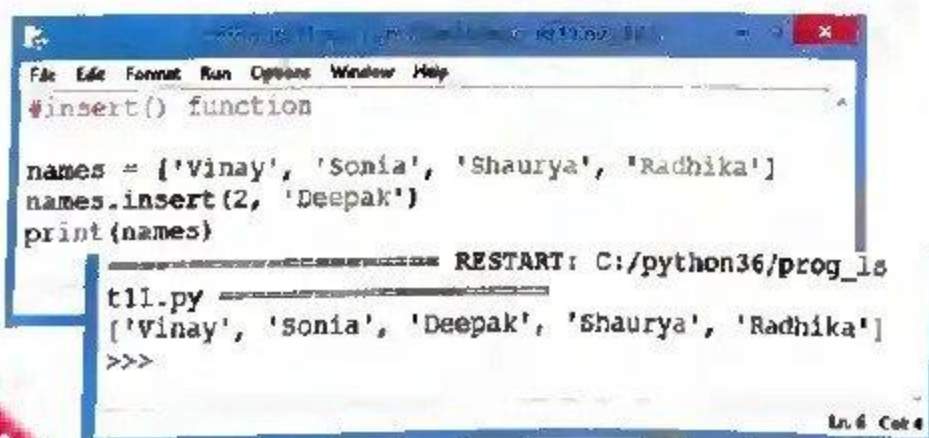
where,

`list_name` is the name of the list

`index_number` is the index where the new value is to be inserted

`value` is the new value to be inserted in the list

Example 7:



```
#insert() function

names = ['Vinay', 'Sonia', 'Shaurya', 'Radhika']
names.insert(2, 'Deepak')
print(names)

===== RESTART: C:/python36/prog_ls
t11.py
['Vinay', 'Sonia', 'Deepak', 'Shaurya', 'Radhika']
>>>
```

Example 8:

```
>>> l1 = [10, 20, 30, 40]
>>> l1.append([50, 60])
>>> l1
[10, 20, 30, 40, [50, 60]]
>>> list1 = [10, 20, 30, 40, 50]
>>> list1.insert(2, 25)
>>> list1
[10, 20, 25, 30, 40, 50]
>>> list1.insert(0, 5)
>>> list1
[5, 10, 20, 25, 30, 40, 50]
```

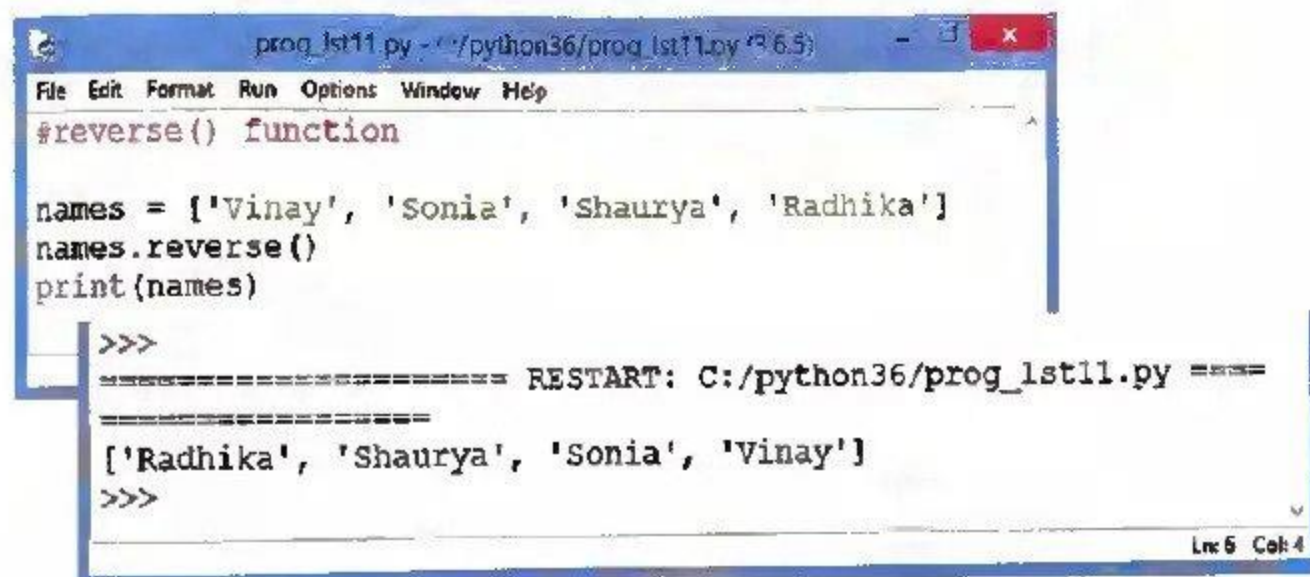
8.6.4 reverse()

The `reverse()` function in Python reverses the order of the elements in a list. This function reverses the item of the list. It replaces a new value "in place" of an item that already exists in the list, i.e., it does not create a new list.

Example 9:

```
>>> list1 = [24, 56, 10, 99, 28, 90]
>>> list1.reverse()
>>> list1
[90, 28, 99, 10, 56, 24]
```

Example 10:



The screenshot shows a Python IDE window titled 'prog_lst11.py - C:/python36/prog_lst11.py (3.6.5)'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The script content is as follows:

```
#reverse() function

names = ['Vinay', 'Sonia', 'Shaurya', 'Radhika']
names.reverse()
print(names)
```

The execution output in the console shows a restart message and the reversed list:

```
>>>
===== RESTART: C:/python36/prog_lst11.py =====
['Radhika', 'Shaurya', 'Sonia', 'Vinay']
>>>
```

The status bar at the bottom right indicates 'Line 6 Col: 4'.

8.6.5 index()

The `index()` function in Python returns the index of first matched item from the list. It returns the first occurrence of an item for which the index position is to be searched for in the list. If the element is not present, `ValueError` is generated.

Syntax:

```
List.index(<item>)
```

Example 11:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
>>> list=[12,14,15,17,14,18]
>>> list.index(15)
2
>>> x = 'python'
>>> print(x.index('t'))
2
>>> x = ['India','my','country']
>>> print(x.index('India'))
0
>>>
```

8.6.6 Updating List

Lists are mutable; you can assign new value to existing value. We can use assignment operator (=) to change an item or a range of items.

Syntax:

```
List[index]=<new value>
```

Example 12:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
>>> List=[1,2,3]
>>> List[1]=4
>>> print(List)
[1, 4, 3]
>>>
```

Example 13:

We can also create a new list from an existing list using list-slicing.

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more
>>> L1=[10,20,30,50,60]
>>> L2=L1[1:4]
>>> print(L2)
[20, 30, 50]
>>>
```

Example 14:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1=["red","pink","orange","yellow"]
>>> L1[2]="white"
>>> print(L1)
['red', 'pink', 'white', 'yellow']
>>>
```

8.6.7 len()

The len() function returns the length of the list, i.e., the number of elements in a list.

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1 = [5,'book',4,5,'new']
>>> print(len(L1))
5
```

8.6.8 sort()

This function sorts the items of the list, by default in ascending/increasing order. This modification is done "in place", i.e., it does not create a new list.

Syntax:

```
List.sort()
```

Example 15:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1=[10,4,20,98,2,100]
>>> L1.sort()
>>> print(L1)
[2, 4, 10, 20, 98, 100]
```

Example 16:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1=['RED','YELLOW','BLUE','WHITE']
>>> L1.sort()
>>> print(L1)
['BLUE', 'RED', 'WHITE', 'YELLOW']
```

For string elements in a list, sorting is done on the basis of their ASCII values. Python starts by comparing the first element from each element. If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are really big).

It sorts the string in a lexicographic manner. If we want to sort the list in decreasing order, we need to write:

```
>>> list1.sort(reverse=True)
```

```
#sort() method-descending order

names=['Anshul','Pihu','Sarthak','Reema']
names.sort(reverse = True)
print(names)

>>>
RESTART: C:/python36/prog_lst13.p
['Sarthak', 'Reema', 'Pihu', 'Anshul']
>>>
```

sorted()-This function also sorts the items of the list like **sort()** function.

Syntax: **sorted(list, reverse = True/False)**

```
>>> list1 = [4,2,1,9,3,4,6,8,5]
>>> sorted(list1)
[1,2,3,4,4,5,6,8,9]
>>> sorted(list1, reverse=True)
[9,8,6,5,4,4,3,2,1]
>>> sorted(list1, reverse=False)
[1,2,3,4,4,5,6,8,9]
```

Note: **sorted()** function works in the same way as **sort()** function, the only difference being that it returns a new list rather than modifying the list passed as a parameter.

8.6.9 clear()

The **clear()** method removes all items from the list. This method doesn't take any parameters. The **clear()** method only empties the given list. It doesn't return any value.

Example 17:

```
python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1=[10,20,30,40]
>>> L1.clear()
>>> print(L1)
```

8.6.10 count()

The count() method counts how many times an element has occurred in a list and returns it. The syntax of count() method is:

```
list.count(element)
```

Example 18:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> L1=[10,20,10,30,40,10]
>>> L1.count(10)
3
```

Example 19:

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
>>> L1=['a', ('x','y'), [1,2], 'b', 10, [1,2]]
>>> L1.count([1,2])
2
>>> L1.count(('x','y'))
1
>>> L1.count(100)
0
```

8.7 DELETION OPERATION

1. Python provides operator for deleting/removing an item from a list. There are many methods for deletion: If index is known, you can use pop() or del statement.
2. If the element is known, not the index, remove() can be used.
3. To remove more than one element, del() with list slice can be used.

➤ pop():

It removes the element from the specified index and also returns the element which was removed.

Syntax:

```
List.pop (index)
```

Example 20:

```
>>> L1 = [1, 2, 5, 4, 70, 10, 90, 80, 50]
>>> L1.pop(1)      # here the element deleted will be returned
>>> 2              # 2 gets deleted
>>> print(L1)
[1, 5, 4, 70, 10, 90, 80, 50]
```

If no index value is provided in pop(), then the last element from the list is deleted.

```
>>> L1.pop() ----- 50
```

CTM: If no index value is provided in pop(), then the last element is deleted.

pop() can be used with negative index value also.

```
>>> L1=[100, 200, 300, 500]
>>> L1.pop(-1)
500
```

➤ del statement

del statement removes the specified element from the list but does not return the deleted value.

```
>>> L1=[100, 200, 300, 400, 500]
>>> del L1[3]
>>> print(L1)
[100, 200, 300, 500]
```

Note: If an out of range index is provided with del() and pop(), the code will result in runtime error.

```
>>> L1=[100,200,300,400,500]
>>> del L1[5]
```

Traceback (most recent call last):

```
File "<pyshell#31>", line 1, in <module>
    del L1[5]
```

IndexError: list assignment index out of range

del can be used with negative index value also.

```
>>> L1=[100,200,300,400,500]
>>> del L1[-2]
>>> print(L1)
[100, 200, 300, 500]
```

➤ del() with slicing

Consider the following example:

```
>>> L1=[10,20,30,40,50]
>>> del L1[2:4]
>>> print(L1)
[10, 20, 50]
```

The above del statement shall remove 2nd and 3rd elements from the list. As we know that slice selects all the elements up to 4th index, so 4th element will remain in the list.

> remove()

The `remove()` function is used when we know the element to be deleted, not the index of the element.

```
>>> L1 = [1, 2, 5, 4, 70, 10, 90, 80, 50]
>>> L1.remove(90)
```

The above statement will remove the value 90 from the list.

```
>>> print(L1)
[1, 2, 5, 4, 70, 10, 80, 50]
```

Practical Implementation-3

Write a Python code to delete all odd numbers and negative numbers from a given numeric list.

```
#Program to delete all odd numbers and negative numbers in a numeric list
list1 = [11,-1,22,-3,33,55,44,-50,46,101,77,-100,42]
len1 = len(list1)
i = 0
while i < len1:
    if (list1[i] < 0):
        del list1[i]
        len1 = len1 - 1
        i = i - 1
    elif (list1[i] % 2 != 0):
        del list1[i]
        len1 = len1 - 1
        i = i - 1
    i = i + 1
print("List after deletion :",list1)
```

*****Output of the program*****

```
>>>
RESTART: C:/Users/preeti/AppData/Local
1.py
List after deletion : [22, 44, 46, 42]
```

Table 8.1: A few List Methods in a Nutshell

Method	Result
<code>append(item)</code>	Add <i>item</i> to the end of the list.
<code>index(item)</code>	Returns the index of the first element whose value is equal to <i>item</i> . A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>insert(index, item)</code>	Inserts <i>item</i> into the list at the specified <i>index</i> . When an item is inserted into a list, the list is expanded in size to accommodate the new item. The item that was previously at a specified index and all the items after it are shifted by one position towards the end of the list. No exceptions will occur if you specify an invalid index. If you specify an index beyond the end of the list, the item will be added to the end of the list. If you use a negative index that specifies an invalid position, the item will be inserted at the beginning of the list.
<code>sort()</code>	Sorts the items in the list so they appear in ascending order (from the lowest value to the highest value).
<code>remove(item)</code>	Removes the first occurrence of <i>item</i> from the list. A <code>ValueError</code> exception is raised if <i>item</i> is not found in the list.
<code>reverse()</code>	Reverses the order of the items in the list.

➤ Searching List

Lists can easily be searched for values using the `index()` method that expects a value that is to be searched. The output is the index at which the value is kept.

Syntax:

```
list_name.index(element)
```

Example 21:

```
>>> list1=[100, 200, 50, 400, 500, 'Raman', 100, 'Ashwin']
>>> list1.index(400)
3
```

Explanation:

Here we are searching list1 for value 400. The answer is 3 as the element 400 is present at the index value 3.

If the value, we are searching in a list is not found, then an error is displayed.

Example 22:

```
>>> list1=[100, 200, 50, 400, 500, 'Raman', 100, 'Ashwin']
>>> list1.index(100)
0
```

This method only returns the first occurrence of the matching element. Here, the element 100 is present at 0th and 6th index, but it returns 0th index.

Example 23:

```
>>> list1=[100, 200, 50, 400, 500, 'Raman', 100, 'Ashwin']
>>> list1.index(700)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#59>", line 1, in <module>
```

```
list1.index(700)
```

```
ValueError: 700 is not in list
```

Example 24:

To find out whether an element is present in a list or not, we can do the following sequence of operations:

```
>>> list1 = [100, 200, 50, 400, 500, 'Raman', 100, 'Ashwin']
>>> 500 in list1
True
```

Explanation:

So, from the above example it is evident that 'True' was returned as '500' was present in list1.

➤ max()

Returns the element with the maximum value from the list.

Note: To use the `max()` function, all values in the list must be of the same type.

Example 25:

```
>>> list3=[10,20,30,40]
>>> max(list3)
40
```

Example 26:

```
>>> list4=['A', 'a', 'B', 'C']
>>> max(list4)
'a'
```

Here, it will return the alphabet with the maximum ASCII value.

Example 27:

```
>>> list5=['ashwin', 'bharat', 'shelly']
>>> max(list5)
'shelly'
```

Here, it will return the string which starts with character having the highest ASCII value.

Example 28:

```
>>> list5=['ashwin', 'bharat', 'shelly', 'surpreet']
>>> max(list5)
'surpreet'
```

If there are two or more strings which start with the same character, then the second character is compared and so on.

➤ min()

Returns the element with the minimum value from the list.

Note: To use the min() function, all values in the list must be of the same type.

Example 29:

```
>>> list3=[10, 20, 30, 40]
>>> min(list3)
10
```

Example 30:

```
>>> list4=['A', 'a', 'B', 'C']
>>> min(list4)
'A'
```

Here, it will return the alphabet with the minimum ASCII value.

Example 31:

```
>>> list5=['ashwin', 'bharat', 'shelly']
>>> min(list5)
'ashwin'
```

Here, it will return the string which starts with character having the smallest ASCII value.

Example 32:

```
>>> list5=['ashwin', 'bharat', 'shelly', 'surpreet']
>>> min(list5)
'ashwin'
```

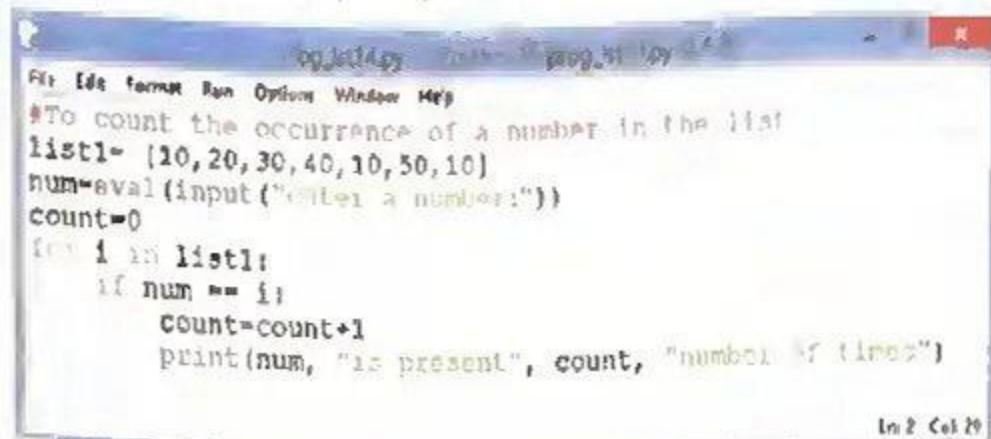
If there are two or more strings which start with the same character, then the second character is compared and so on.



Practical Implementation-4

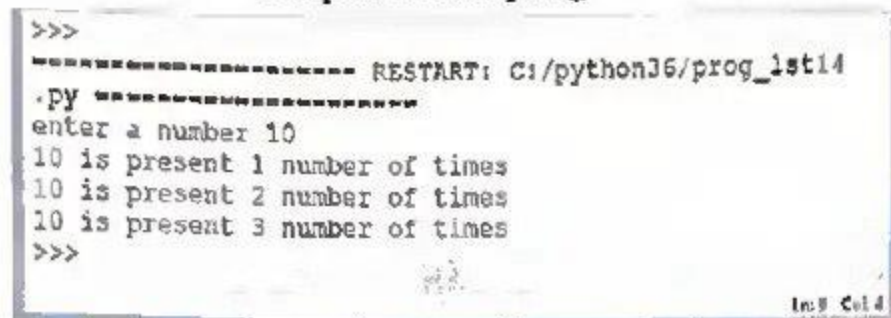
Write a Python script to input a number and count the occurrences of that number in a given list

```
list1=[10, 20, 30, 40, 10, 50, 10]
```



```
File Edit Format Run Options Window Help
#To count the occurrence of a number in the list
list1=[10,20,30,40,10,50,10]
num=eval(input("enter a number:"))
count=0
for i in list1:
    if num == i:
        count=count+1
        print(num, "is present", count, "number of times")
```

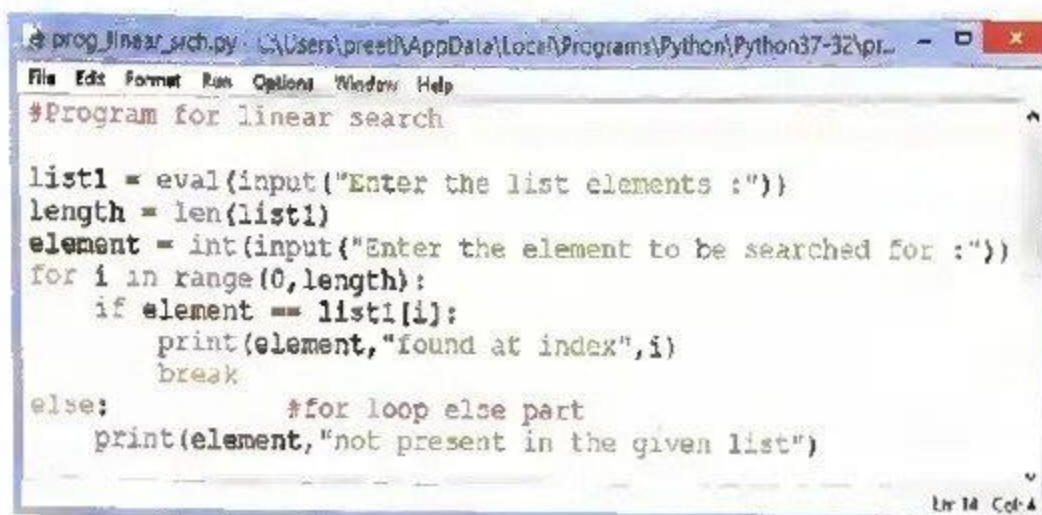
*****Output of the program*****



```
>>>
----- RESTART: C:/python36/prog_1st14
.py -----
enter a number 10
10 is present 1 number of times
10 is present 2 number of times
10 is present 3 number of times
>>>
```

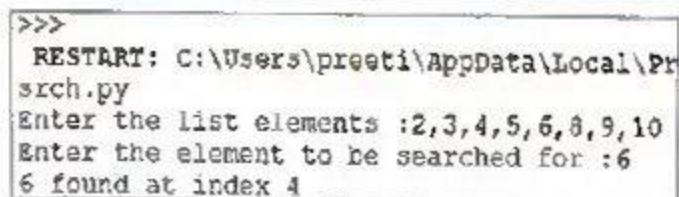
Practical Implementation-5

Write a Python script to input a number and perform linear search.



```
@ prog_linear_srch.py C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\pr...
File Edit Format Run Options Window Help
#Program for linear search

list1 = eval(input("Enter the list elements :"))
length = len(list1)
element = int(input("Enter the element to be searched for :"))
for i in range(0,length):
    if element == list1[i]:
        print(element,"found at index",i)
        break
else:
    #for loop else part
    print(element,"not present in the given list")
```



```
>>>
RESTART: C:\Users\preeti\AppData\Local\Pr
srch.py
Enter the list elements :2,3,4,5,6,8,9,10
Enter the element to be searched for :6
6 found at index 4
```

Practical Implementation-6

The temperature of a week is given in WTemp list in the form of day and its temperature values are as follows:

```
WTemp = ['Mon', 45, 'Tue', 43, 'Wed', 42, 'Thu', 40, 'Fri', 38, 'Sat', 40,
         'Sun', 38]
```

Using the above list, write a program to create and print two separate lists as given below:

```
Days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

```
Degrees = [45, 43, 42, 40, 38, 40, 38]
```

```

#Program to split original list into two separate lists
WTemp = ['Mon', 45, 'Tue', 43, 'Wed', 42, 'Thu', 40, 'Fri', 38, 'Sat', 40, 'Sun', 38]
ln = len(WTemp)
Days = []
Degrees = []
for i in range(ln):
    if (i % 2) == 0:
        Days.append(WTemp[i])
    else:
        Degrees.append(WTemp[i])
print("Original list is: ", WTemp)
print("The days are: ", Days)
print("The temperatures are: ", Degrees)

```

```

>>>
RESTART: C:/Users/reeti/AppData/Local/Programs/Python/Python37-32/prog_split_1
ist1.py
Original list is: ['Mon', 45, 'Tue', 43, 'Wed', 42, 'Thu', 40, 'Fri', 38, 'Sat',
40, 'Sun', 38]
The days are: ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
The temperatures are: [45, 43, 42, 40, 38, 40, 38]

```

8.8 SORTING

Sorting is to arrange the list in an ascending or descending order. Sorting algorithm specifies the way to arrange data in a particular order.

The importance of sorting lies in the fact that data searching can be optimized to a very high level if data is stored in a sorted manner. Sorting is also used to represent data in more readable formats. There are various sorting techniques that can be used to sort the list in ascending or descending order. Here we will discuss two techniques:

1. Bubble Sort
2. Insertion Sort

8.8.1 Bubble Sort

Bubble Sort is one of the simplest sorting techniques where it traverses the whole list by comparing its adjacent elements, sorting them and swapping the elements until the whole list is sorted.

Algorithm

1. START
2. Read the array elements and store it in list `a[]`.
3. Store the length in a variable `n`.
4. For `i=0` to `n-1`, repeat steps 5 and 6.
5. For `j=0` to `n-i-1`, repeat step 4.
6. If `a[j] > a[j+1]`, then swap elements.
7. Display the sorted list.
8. END

Steps:

1. Bubble sort algorithm starts by comparing the first two elements in the list. If and only if the first element is greater than the second, these elements will be swapped. This ensures that the greater of the first two elements is on the second position. In the next step, the second element is compared with the third element, and following the same logic, they are swapped if and only if the second element is greater than the third element. With these

two steps, it is ensured that the greatest of the first 3 elements is on the third position. If this process is repeated until the last element, the greatest element will end up on the last position in the list. This completes the first step in Bubble Sort.

2. After the first step gets completed, the largest element in array will get placed in the last position of the list.
3. If there are n elements to be sorted, then the process mentioned above should be repeated $n-1$ times to get the required result.
4. For performance optimization, in the second step, the last and the second last elements are not compared since this comparison was already done in the previous step, and the last element is already present in its correct position.
5. Similarly, in the third step, second last and third last elements are not compared and so on.

Let us see this with the help of an example.

Sorting in ascending order using bubble sort:

Suppose list is [42, 29, 74, 11, 65, 58]

First pass

[42, 29, 74, 11, 65, 58] $\rightarrow$ [29, 42, 74, 11, 65, 58] it will swap since $29 < 42$
[29, 42, 74, 11, 65, 58] $\rightarrow$ [29, 42, 74, 11, 65, 58] no swapping as $42 < 74$
[29, 42, 74, 11, 65, 58] $\rightarrow$ [29, 42, 11, 74, 65, 58] it will swap since $11 < 74$
[29, 42, 11, 74, 65, 58] $\rightarrow$ [29, 42, 11, 65, 74, 58] it will swap since $65 < 74$
[29, 42, 11, 65, 74, 58] $\rightarrow$ [29, 42, 11, 65, 58, 74] it will swap since $58 < 74$

The largest element is settled at its correct place, i.e., at the end. So, the first five elements form the unsorted part while the last element forms the sorted part and will be compared for the next pass.

Second pass

[29, 42, 11, 65, 58, 74] $\rightarrow$ [29, 42, 11, 65, 58, 74] no swapping as $29 < 42$
[29, 42, 11, 65, 58, 74] $\rightarrow$ [29, 11, 42, 65, 58, 74] it will swap since $11 < 42$
[29, 11, 42, 65, 58, 74] $\rightarrow$ [29, 11, 42, 65, 58, 74] no swapping as $65 > 42$
[29, 11, 42, 65, 58, 74] $\rightarrow$ [29, 11, 42, 58, 65, 74] it will swap since $58 < 65$

The next largest element is settled at its correct place. So, the first four elements form the unsorted part while the last two elements form the sorted part and will be compared for the next pass.

Third pass

$[29, 11, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ it will swap since $29 > 11$
 $[11, 29, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ no swapping as $29 < 42$
 $[11, 29, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ no swapping as $42 < 58$

The next largest element is settled at its correct place. So, the first three elements form the unsorted part while the last three elements form the sorted part.

Fourth pass

$[11, 29, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ no swapping as $29 > 11$
 $[11, 29, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ no swapping as $42 > 29$

The next largest element is settled at its correct place. So, the first two elements form the unsorted part while the last four elements form the sorted part.

Fifth pass

$[11, 29, 42, 58, 65, 74] \rightarrow [11, 29, 42, 58, 65, 74]$ no swapping as $11 < 29$

The next largest element is settled at its correct place. And finally the list is sorted.

Here, the total number of items, say n , is 6, so outer loop will be executed $n-1$ times, i.e., 5 times. There will be 5 passes through the list.

For each pass or outer loop iteration, it compares adjacent items and swaps those that are not in order. Each pass places the next largest value in its correct place and there is no need to compare these elements as they are already placed at their correct place.

Code:

```
prog3_chap7bubblesort.py - C:/Users/preeti/AppData/Local/Programs/Python/Python26
File Edit Format Run Options Window Help
# Bubble Sort
def main():
    l = [42, 29, 74, 11, 65, 58]
    n = len(l)
    print("Original list: ", l)
    for i in range(n-1):
        for j in range(n-i-1):
            if l[j] > l[j+1]:
                l[j], l[j+1] = l[j+1], l[j]
    print("List after sorting is: ", l)
```

*****Output of the program*****

```

File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/prog3_chap7b
ubblesort.py
>>> main()
Original list: [42, 29, 74, 11, 65, 58]
List after sorting is: [11, 29, 42, 58, 65, 74]
>>>

```

Practical Implementation-7

Write an interactive program to create an integer list of 8 integers and sort the list in ascending order using Bubble Sort method.

```

#Program to create an integer list of 8 integers and
#sort the list in ascending order using Bubble Sort method.
l1 = list()
#l1 = [22,12,4,18,45,98,42,80]
ch = 'y'
n2 = 1
print("Enter the list elements: ")
while ch == 'y' or ch == 'Y':
    print("element is:",n2,end=' ')
    val=int(input())
    l1.append(val)
    print("Do you want to add more elements <y/n?:",end=' ')
    ch =input()
    if ch == 'n' or ch == 'N':
        break
    else:
        n2 += 1
l1.append(0)
ctr=i=0
n =len(l1)
print("The Original list is:")
for i in range(0,n):
    print(l1[i],end=' ')
for i in range(n):
    for j in range(n-1):
        if l1[j] > l1[j+1]:
            ctr += 1
            temp = l1[j]
            l1[j] = l1[j+1]
            l1[j+1] = temp
print()
print("The Sorted list")
for i in range(0,n):
    print(l1[i],end=' ')
print("The total no of passes:",ctr)

```

```

>>>
RESTART: C:\Users\preeti\AppData\Local\Programs\Python\
.PY
Enter the list elements:
element is: 1 22
Do you want to add more elements <y/n?: y
element is: 2 12
Do you want to add more elements <y/n?: y
element is: 3 4
Do you want to add more elements <y/n?: y
element is: 4 18
Do you want to add more elements <y/n?: y
element is: 5 45
Do you want to add more elements <y/n?: y
element is: 6 98
Do you want to add more elements <y/n?: y
element is: 7 42
Do you want to add more elements <y/n?: y
element is: 8 80
Do you want to add more elements <y/n?: n
The Original list is:
22 12 4 18 45 98 42 80 0
The Sorted list
0 4 12 18 22 42 45 80 98 The total no of passes: 15

```

In the above program, the elements of list are being inputted by the user interactively, i.e., till the user enters the choice as yes (y or Y). The inputted elements are appended into the list one by one. The next step is to sort this list using bubble sort. The bubble sort method compares the adjacent element of the list $l1$ starting with the first (22) and second (12) elements, followed by swapping (22 being swapped with 12) if they are out of order. Just like the first and second elements, the third and second elements are compared and swapped. This process continues until the end of the list and the output so obtained is the sorted list in ascending order.

8.8.2 Insertion Sort

Consider that you have report cards of 10 students and you have to sort these in ascending order according to roll number.

Well, you will have to go through each report card from the start or the back, and find its right position by comparing its roll number with the roll number on every other report card. Once you find the right position, you will insert the report card there.

Similarly, if more new report cards are provided to you, you can easily repeat the same process and insert the new report cards and get the report cards sorted too.

This is exactly how insertion sort works. It starts from the index 1 (not 0), and each index starting from index 1 is like a new report card that you have to place at the proper position in the sorted sub-array on the left.

Algorithm for insertion sort

Step 1: START

Step 2: Enter the elements in list A, store the length of list in N.

Step 3: For $i=1$ to N, repeat steps 4 to 8, else step 8.

Step 4: $j=A.index(i)$

Step 5: While $(A[j-1]>A[j])$ and $j>0$, repeat steps 6 to 7, else step 3.

Step 6: $A[j-1], A[j] = A[j], A[j-1]$

Step 7: $j=j-1$

Step 8: Print the list

Step 9: END

Let us see this with the help of an example.

Sorting in ascending order using Insertion sort:

Suppose list is [70, 49, 31, 6, 65, 81, 68]

First pass

Sorted array Unsorted array

[70, 49, 31, 6, 65, 81, 68] → [49, 70, 31, 6, 65, 81, 68] it will swap since $70 > 49$

The second element of an array, i.e., 49, is compared with the elements that appear before it, i.e., first element 70. Since $70 > 49$, so swapping will take place and element is inserted at correct position. After first pass, first two elements of an array will be sorted.

Second pass

Sorted elements Unsorted elements

[49, 70, 31, 6, 65, 81, 68] → [49, 31, 70, 6, 65, 81, 68] it will swap since $70 > 31$

[49, 31, 70, 6, 65, 81, 68] → [31, 49, 70, 6, 65, 81, 68] it will further swap since $49 > 31$

The third element of an array, i.e., 31, is compared with the elements that appear before it, i.e., 70. Since $70 > 31$, so it will swap it with 70, then it will compare it with 49. Since $49 > 31$, so swapping will take place and finally 31 will be positioned at its correct place in the sorted part of the array.

Third pass

Sorted elements Unsorted elements

[31, 49, 70, 6, 65, 81, 68] → [31, 49, 6, 70, 65, 81, 68] it will swap since $70 > 6$

[31, 49, 6, 70, 65, 81, 68] → [31, 6, 49, 70, 65, 81, 68] it will swap since $49 > 6$

[31, 6, 49, 70, 65, 81, 68] → [6, 31, 49, 70, 65, 81, 68] it will swap since $31 > 6$

The fourth element of an array, i.e., 6, is compared with the elements that appear before it, i.e., 70 first, then with 49 and 31, and finally placed at its correct place in the sorted part of the array.

Fourth pass

Sorted elements Unsorted elements

[6, 31, 49, 70, 65, 81, 68] → [6, 31, 49, 65, 70, 81, 68] it will swap since $70 > 65$

The fifth element of an array, i.e., 65, is compared with the elements that appear before it, i.e., 70 first, since $70 > 65$, so it will swap it with 70, and further no more comparison will take place because 65 is already placed at its correct position.

Fifth pass

Sorted elements Unsorted elements

[6, 31, 49, 65, 70, 81, 68] → [6, 31, 49, 65, 70, 81, 68] no swapping will take place since $70 < 81$

The sixth element of an array, i.e., 81, is compared with the elements that appear before it, i.e., 70 first. Since $70 < 81$, so it will not swap as this is already placed at its correct position.

Sixth pass

Sorted elements Unsorted elements

[6, 31, 49, 65, 70, 81, 68] → [6, 31, 49, 65, 70, 68, 81] it will swap since $81 > 68$

[6, 31, 49, 65, 70, 68, 81] → [6, 31, 49, 65, 68, 70, 81] it will swap since $70 > 68$

The seventh element of an array, i.e., 68, is compared with the elements that appear before it, i.e., 81 first and then with 70, and gets its correct position in the sorted array.

Now the list is sorted. The total number of items (say n) is 7, so outer loop will be executed $n-1$ times, i.e., 6 times. There will be 6 passes through the list.

For each pass, it not only compares adjacent items and swaps but also slides up each larger element until it gets to the correct location in the sorted part of the array.

Code:

```
Python 3.6.5 (tags/v3.6.5:85c0932b4, Mar 28 2018, 16:07:44) [MSC v.1900
32 bit (Intel)] on win32
File Edit Format Run Debug Options Window Help
# Insertion Sort
a = [70, 49, 31, 6, 65, 81, 68]
print("Original list : ", a)
for i in a:
    j = a.index(i)
    while j > 0:
        if a[j-1] > a[j]:
            a[j-1], a[j] = a[j], a[j-1]
        else:
            break
        j = j-1
    print("List after sorting : ", a)
```

*****Output of the program*****

```
Python 3.6.5 (tags/v3.6.5:85c0932b4, Mar 28 2018, 16:07:44) [MSC v.1900
32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
RESTART: C:/Users/praeti/AppData/Local/Programs/Python/Python36-32
/proga_chap7insertion.py
Original list : [70, 49, 31, 6, 65, 81, 68]
List after sorting : [6, 31, 49, 65, 68, 70, 81]
>>>
```

Steps:

1. Compare the current element in the iteration (say A) with the previous adjacent element to it. If it is in order, then continue the iteration (step 5) else go to step 2.
2. Swap the two elements (the current element in the iteration (A) and the previous adjacent element to it).



3. Compare A with its new previous adjacent element. If they are not in order, then proceed to step 4.
4. Swap if they are not in order and repeat steps 3 and 4.
5. Continue the iteration.

Practical Implementation-8

Write a Python program to shift each element of a list to the right so that the last element becomes the first element. For example, the given list:

2	3	4	5	6
---	---	---	---	---

should become

6	2	3	4	5
---	---	---	---	---

```

#program to shift each element of a list to the right and the last
#element to become the first element

list1 = [2,4,6,8,10]
temp = list1[len(list1)-1]
for i in range(len(list1)-1,-1,-1):
    list1[i]=list1[i-1]
list1[0]=temp
print(list1)
  
```

RESTART: C:/Users/ghr.py
[10, 2, 4, 6, 8]

Application: Insertion Sort

It is efficient for small data sets but very inefficient for large lists. Insertion sort is adaptive which means it reduces its total number of steps if a partially sorted array is provided as input, making it efficient. It does not require additional memory.

Application: Bubble Sort

It is comparatively simpler to understand than some other sorting algorithms. Bubble sort is the fastest and the easiest sorting available under very specific circumstances. It originally became known primarily because it was one of the first algorithms and because of being the fastest on an extremely small and/or nearly sorted set of data. Insertion sort, on the other hand, is a preferred choice for sorting small data sets.

Efficiency of any sorting algorithm is measured by counting the number of algorithm's operations.

Practical Implementation-9

Write a menu-driven program to perform various list operations, such as:

1. Append an element
2. Insert an element
3. Append a list to the given list
4. Modify an existing element
5. Delete an existing element from its position
6. Delete an existing element with a given value
7. Sort the list in ascending order
8. Sort the list in descending order
9. Display the list

```

prog_list_complete.py /Users/pree/V/AppData/Local/Programs/Python/Python37-32/prog_list.py
File Edit Format Run Options Window Help
#Menu driven program to do various list operations
list1 = [22,4,16,38,13] #list1 has 5 elements
choice = 0
while True:
    print("The list has the following elements:", list1)
    print("\n L I S T   O P E R A T I O N S")
    print(" 1. Append an element")
    print(" 2. Insert an element at the desired position")
    print(" 3. Append a list to the given list")
    print(" 4. Modify an existing element")
    print(" 5. Delete an existing element by its position")
    print(" 6. Delete an existing element by its value")
    print(" 7. Sort the list in ascending order")
    print(" 8. Sort the list in descending order")
    print(" 9. Display the list")
    print("10. Exit")
    choice = int(input("ENTER YOUR CHOICE (1-10): "))
    #append element
    if choice == 1:
        element = int(input("Enter the element to be appended: "))
        list1.append(element)
        print("The element has been appended\n")
    #insert an element at desired position
    elif choice == 2:
        element = int(input("Enter the element to be inserted: "))
        pos = int(input("Enter the position:"))
        list1.insert(pos, element)
        print("The element has been inserted\n")
    #append a list to the given list
    elif choice == 3:
        newList = int(input("Enter the list to be appended: "))
        list1.extend(newList)
        print("The list has been appended\n")
    #modify an existing element
    elif choice == 4:
        i = int(input("Enter the position of the element to be modified: "))
        if i < len(list1):
            newElement = int(input("Enter the new element: "))
            oldElement = list1[i]
            list1[i] = newElement
            print("The element",oldElement,"has been modified\n")
        else:
            print("Position of the element is more than the length of list")
    #delete an existing element by position
    elif choice == 5:
        i = int(input("Enter the position of the element to be deleted: "))
        if i < len(list1):
            element = list1.pop(i)
            print("The element",element,"has been deleted\n")
        else:
            print("\nPosition of the element is more than the length of list")
    #delete an existing element by value
    elif choice == 6:
        element = int(input("\nEnter the element to be deleted: "))
        if element in list1:
            list1.remove(element)
            print("\nThe element",element,"has been deleted\n")
        else:
            print("\nElement",element,"is not present in the list")
    #list in sorted order
    elif choice == 7:
        list1.sort()
        print("\nThe list has been sorted")
    #list in reverse sorted order
    elif choice == 8:
        list1.sort(reverse = True)
        print("\nThe list has been sorted in reverse order")
    #display the list
    elif choice == 9:
        print("\n The list is:", list1)
    #exit from the menu
    elif choice == 10:
        break
    else:
        print("Choice is not valid")
        print("\n\nPress any key to continue.....")
        ch = input()

```

*****Output of the program*****

```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Pyth
The list has the following elements [22, 4, 16, 38, 13]
```

LIST OPERATIONS

1. Append an element
2. Insert an element at the desired position
3. Append a list to the given list
4. Modify an existing element
5. Delete an existing element by its position
6. Delete an existing element by its value
7. Sort the list in ascending order
8. Sort the list in descending order
9. Display the list
10. Exit

ENTER YOUR CHOICE (1-10): 1

Enter the element to be appended: 45

The element has been appended

The list has the following elements [22, 4, 16, 38, 13, 45]

LIST OPERATIONS

1. Append an element
2. Insert an element at the desired position
3. Append a list to the given list
4. Modify an existing element
5. Delete an existing element by its position
6. Delete an existing element by its value
7. Sort the list in ascending order
8. Sort the list in descending order
9. Display the list
10. Exit

ENTER YOUR CHOICE (1-10): 2

Enter the element to be inserted: 5

Enter the position: 3

The element has been inserted

The list has the following elements [22, 4, 16, 5, 38, 13, 45]

ENTER YOUR CHOICE (1-10): 8

The list has been sorted in reverse order

The list has the following elements [45, 38, 22, 16, 13, 5, 4]

LIST OPERATIONS

1. Append an element
2. Insert an element at the desired position
3. Append a list to the given list
4. Modify an existing element
5. Delete an existing element by its position
6. Delete an existing element by its value
7. Sort the list in ascending order
8. Sort the list in descending order
9. Display the list
10. Exit

ENTER YOUR CHOICE (1-10): 7

The list has been sorted

The list has the following elements [4, 5, 13, 16, 22, 38, 45]



MEMORY BYTES

- List is a sequence data type.
- A list is a mutable sequence of values which can be of any type and they are indexed by an integer.
- A list is created by placing all the items (elements) inside a square bracket [] separated by commas.
- A list can even have another list as an item. This is called nested list.
- Another way of creating tuple is built-in function list().
- Traversing a list means accessing each element of a list. This can be done by using looping statement either *for* or *while*.

- List slicing allows you to obtain a subset of items.
- `len()` function returns the total number of elements in the list.
- We can combine lists to form a new list. The '+' operator simply performs a concatenation with list.
- The 'in' operator checks whether a given element is contained in a list. It returns true if element appears in the list, otherwise returns false.
- We can compare two lists by using comparison operators, i.e., <, >, ==, != etc.
- We can create a new list from an existing list using list-slicing.
- `append()` method adds a single item at the end of the existing list. It does not return a new list; rather, it modifies the original list.
- The `extend()` adds all the items of a list (passed as an argument) at the end of another list.
- `pop()` function removes the element from the specified index and also returns the element which was removed.
- `del` statement removes the specified element from the list but does not return the deleted value.
- `remove()` function is used when we know the element to be deleted, not the index of the element.
- There are various other list methods available like: `len()`, `mean()`, `index()`, `clear()`, `count()`, `insert()`, `reverse()`, `sort()`, `sorted()`.
- Sorting is to arrange the list in the ascending or descending order. Sorting algorithm specifies the way to arrange data in a particular order.
- Bubble Sort is one of the simplest sorting techniques where it traverses the whole list by comparing its adjacent elements, sorting them and swapping the elements until the whole list is sorted.
- In Insertion Sort, for each pass, the correct position for the current element is found by comparing adjacent elements before the current position and sliding larger elements up until an element less than the current element is found.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) List in Python is classified as a data type.
- (b) A list consists of elements of type.
- (c) Lists are indexed by an data type.
- (d) Elements of a list are enclosed in brackets.
- (e) Lists are, you can update or edit the list.
- (f) method is used to delete elements from a list if index is not known.
- (g) method is used to delete elements from a list if index is known.
- (h) The method adds a single item to the existing list at the end.
- (i) You can compare two lists by using operators.
- (j) The operator replicates a list.
- (k) The operator creates a reference of the original list that shares the same memory location.
- (l) The function returns the deleted element of the list.
- (m) The function arranges the elements of the list either in ascending or descending order.
- (n) is to arrange the list in an ascending or descending order.

2. State whether the following statements are True or False.

- (a) Python lists are mutable.
- (b) Lists are depicted through curly brackets only.
- (c) A list can contain heterogeneous values.
- (d) We can create an empty list by the code `L=list()`

- (e) A list is stored in memory as Integer.
- (f) `[1,2,3,4]<[9,1]` it will give the output as True.
- (g) `List[a:b]` will give the elements between a and b.
- (h) The `append()` is used to add a value in a list at any place.
- (i) `index()` will return the index of first matched item from the list.
- (j) `extend()` function is used to add multiple list items in a list.

3. Multiple Choice Questions (MCQs):

- (a) Consider the following:

```
List=['h', 'o', 'i']
```

The length of the above list is:

- (i) 5 (ii) 3 (iii) 0 (iv) None of these

- (b) The list L1 contains `[3,4,5,6,6,8,0]`

What will be the output of the following expression?

```
print(L1[-1])
```

- (i) 0 (ii) 3 (iii) 5 (iv) 8

- (c) Which of the following statements is correct about List?

- (i) List can contain values of mixed data types. (ii) List can contain duplicate values.
(iii) A list with no elements is called an empty list. (iv) All of these

- (d) Which operator can be used with List?

- (i) in (ii) not in
(iii) both (i) & (ii) (iv) Arithmetic Operators only

- (e) Which of the following commands will create a List?

- (i) `List1=list()` (ii) `List1=[]`
(iii) `List1=[1,2,3,"a"]` (iv) All of these

- (f) What is the use of `append()` function in List?

- (i) It adds an item to the end of the list. (ii) It adds an item at the beginning of the list.
(iii) It adds an item anywhere in the List. (iv) All of these

- (g) Select the output of the following expression:

```
str1="pen"
```

```
print(list(str1))
```

- (i) `['p', 'e', 'n']` (ii) `[pen]` (iii) `[p/e/n]` (iv) `{"pen"}`

- (h) The sequential accessing of each of the elements in a list is called:

- (i) List Indexing (ii) List Traversal (iii) List Slicing (iv) List Accessing

- (i) Consider the following lists:

```
L1=["this", "is", "a", "list"]
```

```
L2=["this", "is", "another list"]
```

Which of the following statements will result in an error?

- (i) `L1=L2` (ii) `L1.copy()`
(iii) `L2.append(1,2,3)` (iv) `L2.pop()`

- (j) Consider the following code:

```
>>>A=[10,20,30,40]
```

```
>>>B=A.copy()
```

```
>>>A[2]=50
```

```
>>>B
```

Which of the following will be the elements of list A and B?

- (i) `A=[10,20,50,40]` (ii) `A=[10,20,30,40]`
`B=[10,20,30,40]` `B=[10,20,50,40]`
 (iii) `A=[10,20,30,40,50]` (iv) `A=[10,20,40]`
`B=[10,20,50,40]` `B=[10,20,50,40]`

(k) Consider the following lists:

A = [1, 2]

B = [1, 2]

What will be the output of the following?

```
print(A==B)
```

(i) True

(ii) False

(iii) Error

(iv) No output

(l) Which of the following function will return the first occurrence of the specified element in a list?

(i) `sort()`

(ii) `value()`

(iii) `index()`

(iv) `Sorted()`

SOLVED QUESTIONS

1. What is a list?

Ans. A list is a mutable sequence of values which can be of any type. Lists are indexed by an integer. Elements of a list are enclosed in **square brackets** `[]`, separated by commas. Lists allow duplicate values as well. The elements of a list can be accessed by its index.

2. What are the differences between lists and strings?

Ans. The differences between lists and strings are:

(a) Strings cannot be changed but lists are mutable. They can be changed.

(b) Lists use square brackets unlike strings which are enclosed in single or double quotes.

(c) String store single type of elements but list can store elements of different types.

3. What is the difference between `extend()` and `append()`?

Ans. The `append()` method adds a single item to the end of the existing list. It doesn't return a new list. Rather, it modifies the original list. The `extend()` adds all multiple items in the form of a list at the end of another list.

4. Can list be used as keys in Python dictionaries? Justify your answer.

Ans. False, because lists are mutable.

5. What is the difference between `pop()` and `remove()`?

Ans. `pop()` function removes the element from the specified index and also returns the element which was removed. If no index value is provided in `pop()`, then the last element is deleted. Function `remove()` is used when we know the element to be deleted but not the index of the element. It does not return the deleted element.

6. What is the difference between insertion sort and bubble sort?

Ans. • Bubble Sort is one of the simplest sorting techniques where it traverses the whole list by comparing its adjacent elements, sorting them and swapping the elements until the whole list is sorted.

• In Insertion Sort, for each pass it not only compares adjacent items and swaps but also slides up each larger element until it gets to the correct location to insert into the sorted part of the array. Insertion sort, on the other hand, is a preferred choice for sorting small data sets.

7. Given below are statements for creating lists. Find errors, if any, and rewrite the correct statements.

(a) `L1 = 1, 6, a, 8`

(b) `L2 = {10}`

(c) `L1 = [[0, 1, 2, 3], ['my', 'book']]`

(d) `L1 = [[0, 1, 2, 3], [4, 5, 6]]`

(e) `L1 = ('a', 'b', 'c', [1, 2, 3, A])`

(f) `L1 = [Aman, Lakshay, Aushim, Nishant]`

Ans. (a) `L1 = [1, 6, 'a', 8]`

(b) `L2 = [10]`

(c) `L1 = [[0, 1, 2, 3], ['my', 'book']]`

(d) `L1 = [[0, 1, 2, 3], [4, 5, 6]]`

(e) `L1 = ['a', 'b', 'c', [1, 2, 3, 'A']]`

(f) `L1 = ['Aman', 'Lakshay', 'Aushim', 'Nishant']`

8. Suppose `L=["abc", [6, 7, 8], 3, 'mouse']`

Consider the above list and answer the following:

- (a) `L[3:]` (b) `L[:2]`
(c) `L[1:2]` (d) `L[1][1]`

Ans. (a) `['mouse']` (b) `['abc', 3]`
(c) `[[6, 7, 8]]` (d) `7`

9. Write the most appropriate list method to perform the following tasks:

- (a) Delete a given element from the list.
(b) Get the position of an item in the list.
(c) Delete the 3rd element from the list.
(d) Add single element at the end of the list.
(e) Add an element in the beginning of the list.
(f) Add elements in the list at the end of a list.

Ans. (a) `remove()` (b) `index()` (c) `pop()/del`
(d) `append()` (e) `insert()` (f) `extend()`

10. Give the output of the following:

```
(i) a=[2,7,9,13,28]
    print(a[2:]+a[:2])
(ii) values = [[3, 4, 5, 1], [33, 6, 1, 2]]
    for row in values:
        row.sort()
        for element in row:
            print(element, end = " ")
        print()
```

Ans. (i) `[9, 13, 28, 2, 7]`
(ii) `1 3 4 5`
`1 2 6 33`

11. Suppose list1 is `[3, 4, 5, 20, 5, 25, 1, 3]`. What is list1 after `list1.pop(1)`?

Ans. `[3, 5, 20, 5, 25, 1, 3]`

12. Given a Python list, find value 20 in the list, and if it is present, replace it with 200. Only update the first occurrence of a value.

`list1 = [5, 10, 15, 20, 25, 50, 20]`

Expected output:

`list1 = [5, 10, 15, 200, 25, 50, 20]`

Ans. `list1 = [5, 10, 15, 20, 25, 50, 20]`

`index1 = list1.index(20)`

`list1[index1] = 200`

`print(list1)`

13. Write a Python program to count the number of strings where the string length is 2 or more and the first and last characters are same from a given list of strings.

Sample List: `['abc', 'xyz', 'cdc', '121']`

Expected Result: 2

Ans. `l=['abc', 'xyz', 'cdc', '121']`

`c=0`

`for word in l:`

`if len(word) > 1 and word[0] == word[-1]:`

`c=c+1`

`print(c)`

14. Write the output of the following:

```
L=[]
L1=[]
L2=[]
for i in range(6,10):
    L.append(i)
for i in range(10,4,-2):
    L1.append(i)
for i in range(len(L1)):
    L2.append(L[i]+L1[i])
L2.append(len(L)-len(L1))
print(L2)
```

Ans. [16, 15, 14, 1]

15. Write the output of the following:

```
(a) str1='book'
    print(list(str1))
(b) L = [2, 4, 5, 6, 2, 3, 4, 4, 7]
    count = L.count(4)
    print(count)
(c) L = [10,45,2,20,30,40]
    L.sort()
    print(L)
    L.reverse()
    print(L)
(d) L = ['p', 'w', 'r', 'd', 'a']
    L.remove('r')
    print(L)
    print(L.pop())
    del L[1]
    print(L)
(e) L1 = [10, 20, 30, 40]
    L1.clear()
    print(L1)
```

Ans. (a) ['b', 'o', 'o', 'k']
(b) 3
(c) [2, 10, 20, 30, 40, 45]
[45, 40, 30, 20, 10, 2]
(d) ['p', 'w', 'd', 'a']
a
['p', 'd']
(e) []

16. Write the output of the following:

```
L1=[500,600]
L2=[35,45]
L1.append(700)
L1.extend(L2)
L1.insert(5,2)
print(L1)
print(L1+L2)
print(L1)
print(L1.index(35))
print(L2*2)
```

Ans. [500, 600, 700, 35, 45, 2]
[500, 600, 700, 35, 45, 2, 35, 45]
[500, 600, 700, 35, 45, 2]
3
[35, 45, 35, 45]

17. Write the output of the following code:

```
List1=[2,4,6,8,10]
List2=List1
List3=List1.copy()
List1[4]=12
List2[3]=14
List1.remove(2)
print("List1",List1)
print("List2",List2)
print("List3",List3)
```

Ans. List1 [4, 6, 14, 12]
List2 [4, 6, 14, 12]
List3 [2, 4, 6, 8, 10]

18. Write a program to arrange a list on integer elements in ascending order using bubble sort technique.
10,51, 2, 18, 4, 31, 13, 5, 23, 64, 29

Ans. #Bubble Sort Implementation

```
num = [10, 51, 2, 18, 4, 31, 13, 5, 23, 64, 29]
ctr = i = 0
n = len(num)
print("The Original list is:")
for i in range(0,n):
    print(num[i], end=' ')
#bubble sort
for i in range(n):
    for j in range(n-1):
        if (num[j] > num[j+1]):
            ctr += 1
            tmp = num[j]
            num[j] = num[j+1]
            num[j+1] = tmp
```

```
print()
print("The sorted list is:")
for i in range(0,n):
    print(num[i], end=' ')
```

Output:

The Original list is:
10 51 2 18 4 31 13 5 23 64 29
The sorted list is:
2 4 5 10 13 18 23 29 31 51 64

19. Write a program to perform linear search on the given list:

```
list = [10, 51, 2, 18, 4, 31, 13, 5, 23, 64, 29]
```

Ans. #Linear Search Implementation

```
num = [10, 51, 2, 18, 4, 31, 13, 5, 23, 64, 29]
pos = 0
print("List elements are: ", end=' ')
for i in num:
    print(i, end=' ')
print()
find = int(input("Enter the element to search: "))
flag = 0
```

```

for i in num:
    if (i == find):
        flag = 1
        pos = num.index(i)
        break
if flag == 1:
    print("Element found at index:", pos)
else:
    print("Element not found")

```

Output:

List elements are: 10 51 2 18 4 31 13 5 23 64 29

Enter the element to search: 18

Element found at index: 3

20. Write the output of the following:

(a) `L=[1,2,3,4,5,6,7,8,9,10]`

`S=[i for i in L if i%2==0]`

`print(S)`

(b) `A=[1,2,3,4]`

`B=[value*3 for value in A]`

`print(B)`

Ans. (a) `[2, 4, 6, 8, 10]`

(b) `[3, 6, 9, 12]`

21. Write a program to shift every element of a list to circularly right, e.g.,

INPUT : 1 2 3 4 5 6 7

OUTPUT : 4 5 6 1 2 3 7

Ans. `l=[1,2,3,4,5,6,7]`

`mid=int(len(l)/2)`

`for i in range(mid):`

`l[i], l[mid+i]=l[mid+i], l[i]`

`print(l)`

22. What will be the status of the following list after fourth pass of bubble sort arranging it in descending order?

95, -1, 38, 2, 100, 65, 78

Ans. Bubble sort

`[95, -1, 38, 2, 100, 65, 78]`

`[95, 38, 2, 100, 65, 78, -1]` – 1st

`[95, 38, 100, 65, 78, 2, -1]` – 2nd

`[95, 100, 65, 78, 38, 2, -1]` – 3rd

`[100, 95, 78, 65, 38, 2, -1]` – 4th

23. What will be the status of the following list after each pass of insertion for arranging elements in descending order?

22, 54, 12, 90, 55, 78

Ans. Insertion sort

`[54, 22, 12, 90, 55, 78]` 1st

`[54, 22, 12, 90, 55, 78]` 2nd

`[90, 54, 22, 12, 55, 78]` 3rd

`[90, 55, 54, 22, 12, 78]` 4th

`[90, 78, 55, 54, 22, 12]` 5th



24. Write a program which accepts a number from the user and prints the frequency of the number in the list LST given as under, if number is not in the LST, it should print "Number not available".

LST = [3, 21, 5, 6, 3, 8, 21, 6]

Ans.

```
LST = eval(input("Enter List:"))
length = len(LST)
element = int(input("Enter element:"))
c = 0
for i in range(0, length):
    if element == LST[i]:
        c = c + 1
if c == 0:
    print("Not found")
else:
    print(c)
```

Output:

Element	Frequency
3	2
21	2
5	1
6	2
8	1

25. Write a program in Python to find the maximum and minimum elements in the list entered by the user.

Ans.

```
lst = []
num = int(input('How many numbers: '))
for n in range(num):
    numbers = int(input('Enter number: '))
    lst.append(numbers)
print("Maximum element in the list is :", max(lst))
print("Minimum element in the list is :", min(lst))
```

Output:

```
How many numbers: 5
Enter number: 1
Enter number: 88
Enter number: 45
Enter number: 27
Enter number: 10
Maximum element in the list is : 88
Minimum element in the list is : 1
```

26. Write the output of the following Python program code:

```
STR1 = list("SNo4")
for i in range(len(STR1)):
    if i == 3:
        x = int(STR1[i])
        x += x - 3
        STR1[i] = x
    elif (STR1[i].islower()):
        STR1[i] = STR1[i].upper()
    else:
        STR1[i] = STR1[i] * 2
print(STR1)
```

Ans. ['SS', 'NN', 'O', '3']

24. Write a program which accepts a number from the user and prints the frequency of the number in the list LST given as under, if number is not in the LST, it should print "Number not available".

LST = [3, 21, 5, 6, 3, 8, 21, 6]

Ans.

```
LST = eval(input("Enter List:"))
length = len(LST)
element = int(input("Enter element:"))
c = 0
for i in range(0, length):
    if element == LST[i]:
        c = c + 1
if c==0:
    print("Not found")
else:
    print(c)
```

Output:

Element	Frequency
3	2
21	2
5	1
6	2
8	1

25. Write a program in Python to find the maximum and minimum elements in the list entered by the user.

Ans.

```
lst = []
num = int(input('How many numbers: '))
for n in range(num):
    numbers = int(input('Enter number: '))
    lst.append(numbers)
print("Maximum element in the list is :", max(lst))
print("Minimum element in the list is :", min(lst))
```

Output:

```
How many numbers: 5
Enter number: 1
Enter number: 88
Enter number: 45
Enter number: 27
Enter number: 10
Maximum element in the list is : 88
Minimum element in the list is : 1
```

26. Write the output of the following Python program code:

```
STR1= list("SNo4")
for i in range(len(STR1)):
    if i==3:
        x=int(i)
        x+=x-3
        STR1[i]=x
    elif (STR1[i].islower()):
        STR1[i] = STR1[i].upper()
    else:
        STR1[i]=STR1[i]*2
print(STR1)
```

Ans. ['SS', 'NN', 'O', '3']

27. Write the output of the following Python program code:

```
L = "neat"
X = ""
L1 = []
count=1
for i in L:
    if i in ['a','e','i','o','u']:
        X = X + i.swapcase()
    else:
        if (count%2)!=0:
            X = X + str((len(L[:count])))
        else:
            X = X + i
        count = count + 1
print(X)
```

Ans. 1EAt

28. WAP to accept integer array and search a particular value in the array. If it exists, then the function should display "exists", otherwise should display "Does not exist".

Ans. L=[10,20,3,100,65,87,2]

```
N=int(input("Enter the number to be searched:"))
for i in range(len(L)):
    if type(L[i])==int and L[i]==N:
        print("Exists at position:",i+1)
        break
```

```
else:
    print("Does not exist")
```

Output:

```
>>> Enter the number to be searched :75
Does not exist
>>> Enter the number to be searched :100
Exists at position :4
```

29. WAP that takes a list 'L' as an argument, adds 5 in all the odd values and 10 in all the even values of the list L. Also display the list.

Input: L=[10,20,3,100,65,87,1]

Output: [20,30,8,110,70,92,12]

Ans. L=[10,20,3,100,65,87,2]

```
for i in range(len(L)):
    if type(L[i])==int:
        if L[i]%2==0:
            L[i]=L[i]+10
        else:
            L[i]=L[i]+5
```

```
print(L)
```

Output:

```
[20, 30, 8, 110, 70, 92, 12]
```

30. WAP to display unique and duplicate items of a given list into two different lists.

Input: L1=[2,7,1,4,9,5,1,4,3]

Output: [2, 7, 1, 4, 9, 5, 3]

[1, 4]

Ans. L1=[2,7,1,4,9,5,1,4,3]

L2=[]

L3=[]

for i in L1:

if i not in L2:

L2.append(i)

else:

L3.append(i)

print(L2)

print(L3)

Output:

[2, 7, 1, 4, 9, 5, 3]

[1, 4]

31. WAP to exchange first-half elements of list with second-half elements assuming list is having even number of elements.

Ans. L=[10,20,30,40,50,60,70]

x=int(len(L)/2)

for i in range(x):

L[i],L[x+i]=L[x+i],L[i]

print(L)

Output:

[40, 50, 60, 10, 20, 30, 70]

32. Consider a nested list

L1=[1,2,3,["computer","device"],4,5,["forty","sixty"],6,7,8] and predict the outputs for the following statements:

(i) L1[2]

(ii) L1[3:6]

(iii) L1[3][0]

(iv) L1[3][0][0:4]

(v) L1[6][1][3]

(vi) L1[3][1]="Memory"

print(L1)

Ans. (i) 3

(ii) [['computer', 'device'], 4, 5]

(iii) 'computer'

(iv) 'comp'

(v) 't'

(vi) [1, 2, 3, ['computer', 'Memory'], 4, 5, ['forty', 'sixty', 6, 7, 8]]

33. Consider a list list1=[1,2,3,4,5].

What would be displayed for the following code:

(i) list1+2

(ii) list1*3

(iii) list1+[10]

Ans. (i) TypeError: can only concatenate list (not "int") to list

(ii) [1, 2, 3, 4, 5, 1, 2, 3, 4, 5, 1, 2, 3, 4, 5]

(iii) [1, 2, 3, 4, 5, 10]

34. WAP to display square of an element if it is an integer and change the case if the element is a string. List L contains both numbers and strings. L = [10, "FUN", 40, "FEW", 50, "FULL"].

Ans. L=[10,"FUN",40,"FEW",50,"FULL"]

for i in range(len(L)):

if type(L[i])==int:

L[i]=L[i]**2

elif type(L[i])==str:

L[i]=(L[i]).swapcase()

print(L)

Output:

[100, 'fun', 1600, 'few', 2500, 'full']

35. WAP in Python, to display those strings which are starting with character 'A' or 'a' from the given list L.

L=["RINKU", "AUSHIM", "VIJAYA", "AKHTAR", "LEENA", "TARUN", "AMAR"]

Ans.

```
L=["RINKU", "AUSHIM", "VIJAYA", "AKHTAR", "LEENA", "TARUN", "AMAR"]
count = 0
for i in L:
    if i[0] in ('aA'):
        count+=1
    print(i)
print("Appearing", count, "times")
```

Output:

AUSHIM

AKHTAR

AMAR

Appearing 3 times

36. WAP to find and display the sum of all the values which are ending with 3 from a list.
e.g., list is:

[33,13,92,99,3,12]

Output will be 49.

Ans.

```
L=[33,13,92,99,3,12]
```

```
sum=0
```

```
x=len(L)
```

```
for i in range(0,x):
```

```
    if type(L[i])==int:
```

```
        if L[i]%10==3:
```

```
            sum+=L[i]
```

```
print(sum)
```

37. A list Num contains the following elements:

3, 21, 5, 6, 14, 8, 14, 3

WAP to swap the content with the next value divisible by 7 so that the resultant list looks like:

3, 5, 21, 6, 8, 14, 3, 14

Ans.

```
num=[3, 21, 5, 6, 14, 8, 14, 3]
```

```
L=len(num)
```

```
i=0
```

```
while i<L:
```

```
    if num[i]%7==0:
```

```
        num[i], num[i+1]= num[i+1], num[i]
```

```
        i=i+2
```

```
    else:
```

```
        i=i+1
```

```
print(num)
```

Output:

[3, 5, 21, 6, 8, 14, 3, 14]

38. WAP to shift the negative numbers to the right and the positive numbers to the left so that the resultant list looks like

Original list: [-12, 11, -13, -5, 6, -7, 5, -3, -6]

Output should be [11, 6, 5, -6, -3, -7, -5, -13, -12]



Ans. `l=[-12, 11, -13, -5, 6, -7, 5, -3, -6]`

```
n=len(l)
print("Original list:",l)
for i in range(n-1):
    for j in range(n-i-1):
        if l[j]<l[j+1]:
            l[j],l[j+1]=l[j+1],l[j]
print("After shifting list is:",l)
```

Output:

Original list: [-12, 11, -13, -5, 6, -7, 5, -3, -6]

After shifting list is: [11, 6, 5, -6, -3, -7, -5, -13, -12]

39. Consider List1=[1,2,3,4,5,6,7,8,9,10]

Write code in Python to:

- (i) Remove last three elements from the list
- (ii) Add element 23 at index value 10
- (iii) Display the list in sorted order.

Ans. (i) `del List1[-3:]` (ii) `List1.insert(10,23)`
(iii) `print (List1.sort())`

UNSOLVED QUESTIONS

1. What are the various ways to create a list?

2. What are the similarities between strings and lists?

3. Why are lists called mutable data type?

4. Define the following functions:

- | | | |
|---------------------------|------------------------------|-----------------------------|
| (i) <code>len()</code> | (ii) <code>append()</code> | (iii) <code>extend()</code> |
| (iv) <code>pop()</code> | (v) <code>remove()</code> | (vi) <code>del</code> |
| (vii) <code>sort()</code> | (viii) <code>sorted()</code> | (ix) <code>copy()</code> |
| (x) <code>count()</code> | (xi) <code>max()</code> | (xii) <code>min()</code> |
| (xiii) <code>sum()</code> | | |

5. What is the difference between `insert()` and `append()` methods of a list?

6. Suppose `L=[10, ["few", "facts", "fun"], 3, 'Good']`

Consider the above list and answer the following:

- | | |
|---------------------------|---------------------------|
| (i) <code>L[3:]</code> | (ii) <code>L[:2]</code> |
| (iii) <code>L[1:2]</code> | (iv) <code>L[1][1]</code> |

7. Find the output of the following:

```
L1=[1, 2, 3]
L2=[4,5,6]
print(L1+list("45"))
print(L1.pop())
L1.remove(2)
print(L1)
L1.extend(L2)
print(L2)
```

8. What will be the output of the following statements?

```
(i) list1 = [12,32,65,26,80,10]
    list1.sort()
    print(list1)
```

```

(ii) list1 = [12, 32, 65, 26, 80, 70]
    sorted(list1)
    print(list1)
(iii) list1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    list1[::-2]
    list1[:3] + list1[3:]
(iv) list1 = [1, 2, 3, 4, 5]
    list1[len(list1)-1]

```

9. What will be the output of the following code segment?

```

myList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for i in range(0, len(myList)):
    if i%2 == 0:
        print(myList[i])

```

10. What will be the output of the following code segment?

```

(i) myList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    del myList[3:]
    print(myList)
(ii) myList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    del myList[:5]
    print(myList)
(iii) myList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    del myList[::2]
    print(myList)

```

11. Write the output of the following Python program code:

```

Str2= list("Cam@123*")
for l in range(len(Str2)-1):
    if l==5:
        Str2[l]= Str2[l]*2
    elif (Str2 [l].isupper()):
        Str2 [l] = Str2 [l]*2
    elif (Str2 [l].isdigit()):
        Str2 [l] = 'D'
print(Str2)

```

12. Differentiate between append() and extend() methods of list.

13. Write a program to read a list of n integers (positive as well as negative). Create two new lists, one having all positive numbers and the other having all negative numbers from the given list. Print all three lists.

14. Write a program to find the largest and the second largest elements in a given list of elements.

15. Write a program to read a list of n integers and find their median.

Note: The median value of a list of values is the middle one when they are arranged in order. If there are two middle values, then take their average.

Hint: Use an inbuilt function to sort the list.

16. What is the basic principle of sorting in insertion sort?

17. Which sorting is more efficient—bubble sort or insertion sort? Give reason.

18. Write a program to read a list of elements. Modify this list so that it does not contain any duplicate elements, i.e., all elements occurring multiple times in the list should appear only once.

19. Write a program to create a list of elements. Input an element from the user that has to be inserted in the list. Also, input the position at which it is to be inserted.

20. Write a program to read elements of a list and do the following:
- The program should ask for the position of the element to be deleted from the list and delete the element at the desired position in the list.
 - The program should ask for the value of the element to be deleted from the list and delete this value from the list.
21. WAP that takes an array *S* as an argument and add all the odd values and display the sum.
22. Write a program to calculate mean of a given list of numbers.
23. WAP in Python to delete all duplicate elements in a list.
- For example,
If list is: [5, 2, 4, -5, 12, 2, 7, 4]
After deleting duplicate elements, new list should be: [5, 2, 4, -5, 12, 7]
24. Write a program to calculate the minimum element of a given list of numbers.
25. Write a code to calculate and display total marks and percentage of a student from the given list storing marks of a student.
26. WAP to multiply an element by 2 if it is odd index for a given list containing both numbers and strings.
27. WAP to count the frequency of an element in a given list.
28. WAP to shift elements of a list so that first element moves to the second index and second index moves to the third index, etc., and last element shifts to the first position.
- Suppose list is [10, 20, 30, 40]
After shifting it should look like: [20, 30, 40, 10]
29. An array *Num* contains the following elements:
3, 25, 13, 6, 35, 8, 14, 45
Write a function to swap the content with next value divisible by 5 so that the resultant array looks like:
25, 3, 13, 35, 6, 8, 45, 14
30. The record of a student (Name, Roll No, Marks in five subjects and percentage of marks) is stored in the following list:
`studRecord = ['Rinku', 'A-36', [56, 98, 99, 72, 69], 78.8]`
Write Python statements to retrieve the following information from the list *studRecord*.
- Percentage of the student
 - Marks in the fifth subject
 - Maximum marks of the student
 - Roll No. of the student
 - Change the name of the student from 'Rinku' to 'Rajat'
31. What will be the status of the following list after fourth pass of insertion sort and fourth pass of bubble sort used for arranging elements in ascending order?
12, 14, -54, 64, 90, 24
32. Write the steps to arrange the following elements using insertion sort:
12, 5, 14, 8, 3, 54, 25, 10, 27, 30

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

- A bank is a financial institution involved in borrowing and lending of money. With advancements in technology, online banking, also known as internet banking, allows customers of a bank to conduct a range of financial transactions through the bank's website any time, anywhere.

Two basic operations performed in an account are Deposit and Withdrawal. Write a menu-driven program that accepts either of the two choices of Deposit and Withdrawal, then accepts an amount, performs the transaction and accordingly displays the balance. The program should display the following options:

- Open a savings bank account
- Add a customer
- Deposit money
- Display sorted records
- Close/delete account

Ans.

```
prog_banking_casestudy.py - C:\Users\preeti\AppData\Local\Programs\Python\Python37-32\prog_banking_casestudy...
File Edit Format Run Options Window Help
ch = 0
list = []
while True:
    print("List Menu:")
    print("1. Add a customer")
    print("2. Modify customer details")
    print("3. Delete a customer details")
    print("4. Sort customer list")
    print("5. Display sorted list")
    print("6. Exit")
    ch = int(input("Enter your choice :"))
    if ch == 1:
        print("1. Add amount deposited")
        print("2. Add a customer record")
        ch1 = int(input("Enter choice 1 or 2:"))
        if ch1 == 1:
            amt = int(input("Enter deposited amount:"))
            pos = int(input("Enter position to add : "))
            list.insert(pos, amt)
        elif ch1 == 2:
            list2 = eval(input("Enter added customer list:"))
            list.extend(list2)
    else:
        print("Valid choices are 1 or 2")
        print("Successfully added")
    elif ch == 2:
        pos = int(input("Enter position whose details are to be modified:"))
        new_cust = int(input("Enter new value :"))
        old = list[pos]
        list[pos] = new_cust
        print(old, "modified with", new_cust)
    elif ch == 3:
        print("1. Delete details by position")
        print("2. Delete details by customer number")
        ch1 = int(input("Enter your choice as 1 or 2:"))
        if ch1 == 1:
            pos = int(input("Enter position :"))
            cust = list.pop(pos)
            print(cust, "deleted")
        elif ch1 == 2:
            cust = int(input("Enter element:"))
            pos = list.remove(cust)
            print("Successfully deleted")
    else:
        print("Valid choices are 1 or 2")
    elif ch == 4:
        print("1. Ascending")
        print("2. Descending")
        ch1 = int(input("Enter choice 1 or 2:"))
        if ch1 == 1:
            list.sort()
        elif ch1 == 2:
            list.sort(reverse=True)
        else:
            print("Valid choices are 1 or 2")
    elif ch == 5:
        print(list)
    elif ch == 6:
        break
    else:
        print("valid choice is 1 to 6")
```

```

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/
_casestudy.py
List Menu
1. Add a customer
2. Modify customer details
3. Delete a customer details
4. Sort customer list
5. Display sorted list
6. Exit
Enter your choice : 1
1. Add amount deposited
2. Add a customer record
Enter choice 1 or 2: 1
Enter deposited amount: 40000
Enter position to add : 1
Successfully added

```

2. Vidyarthi Gurukul conducts physical training camps for the physical well-being of its students. This camp is held for one week on the outskirts of the city. The entire group of students are represented as a list. The list stores the roll numbers of the participants for camp activities. As part of this training, the entire stream of students is divided into two halves. While the first group of students goes for physical training, the second group goes for aerobic classes and, in the later half, both these groups are swapped.
- Write a Python program that maintains and displays details about the swapping of activity classes and awards them stars on the completion of each activity.

Ans.

```

prog_cs1_list.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_cs1_
File Edit Format Run Options Window Help
#To divide the students into two halves and interchange their
#groups for alternate activities and award them with stars
rno_list= [1,2,5,10,14,15,22,25,26,8,3,30,45]
L=len(rno_list)
print("List of the students")
for i in range(L//2):
    rno_list[i],rno_list[L//2+i] = rno_list[L//2+i],rno_list[i]
print(rno_list)
print()
print("Students successfully performed all activities")
for i in range(L):
    if i ==L//2:
        print()
    print(rno_list[i],end="**")

```

```

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/
t.py
List of the students
[22, 25, 26, 8, 3, 30, 1, 2, 5, 10, 14, 15, 45]

Students successfully performed all activities
22**25**26**8**3**30**
1**2**5**10**14**15**45**

```